

Machine-Learned Factor Momentum: Timing Equity Factors with Gradient Boosted Trees

Master in Quantitative Finance
Thesis Supervisor: Francesco Nicolai

June 2026

Abstract

Factor momentum has been shown to improve equity factor investing by dynamically timing factor exposures, but traditional implementations rely on simple return-based timing signals. This thesis examines whether machine learning can improve the timing of systematic equity factor returns relative to traditional time-series and cross-section factor momentum strategies.

Using stock-level return and characteristic data, we construct 48 long-short equity factors and evaluate their performance over the 1985–2024 sample period. We implement XGBoost, LightGBM, and CatBoost under both expanding and fixed-window training schemes to forecast one-month-ahead factor returns, and use these forecasts to construct machine-learned TSFM and CSFM portfolios.

MLTSFM strategies achieve annualized Sharpe ratios between 0.69 and 0.89, compared to 0.68 for traditional TSFM, while MLCSFM strategies reach Sharpe ratios between 0.61 and 0.91 compared to 0.62 for CSFM. MLTSFM strategies also generate statistically significant alphas relative to both TSFM and CSFM. A pooled MLTSFM specification performs particularly strongly, partly driven by its net long factor exposure, reaching a pre-transaction-cost Sharpe ratio of 1.56, producing statistically significant alphas relative to traditional factor momentum strategies, and receiving substantial weight in ex-post mean-variance optimal portfolios, including under no-short selling constraints. Overall, the results suggest that machine learning can improve equity factor timing and provide a return source not fully captured by traditional momentum strategies.

Contents

1	Introduction	4
2	Literature review	5
2.1	What is a Factor?	5
2.2	Multifactor Models	5
2.3	Time-Series Momentum	6
2.4	Factor Momentum Everywhere	7
2.4.1	TSFM	7
2.4.2	CSFM	8
2.4.3	Other Results	8
2.5	Machine Learning & Asset Pricing	9
2.5.1	The Virtue of Complexity in Return Prediction	9
2.5.2	Empirical Asset Pricing via Machine Learning	10
2.5.3	Nonlinear Time Series Momentum	10
3	Research methodology	11
3.1	Use of AI Disclosure	11
3.2	Implementing TSFM	11
3.3	Alternative implementation of aggregated TSFM	13
3.4	Machine-Learned Time-Series Factor Momentum	13
3.4.1	Regression Trees & Gradient Boosting	15
3.4.2	Hyperparameters	16
3.4.3	Bayesian Optimization & Hyperparameter Tuning	17
3.4.4	ML model training regime	18
3.5	Performance comparison of results	19
4	Data collection	20
4.1	Differences in Implementation	20
5	Results & Analysis	21
5.1	Overview of Factors	21
5.2	Autocorrelation in Factor Returns	24
5.3	TSFM Performance & Own-Factor Alpha	25
5.4	Boosting	29
5.4.1	Predictive Performance	29

5.4.2	Optimal Hyperparameters	31
5.5	MLTSFM Performance	32
5.5.1	Portfolio Performance	32
5.5.2	Comparison with TSFM	34
5.6	MLCSFM Performance	35
5.6.1	Portfolio Performance	35
5.6.2	Comparison with CSFM	36
5.6.3	Comparison of Momentum Strategies	38
5.7	Pooled MLTSFM/TSFM	40
5.7.1	Performance measures	40
5.7.2	Factor weights	41
5.7.3	Pooled MLTSFM relative performance	44
5.8	Mean-Variance Analysis	47
5.8.1	Correlations	47
5.8.2	Moving Block Bootstrap	48
5.8.3	Tangency Portfolios	49
5.9	Turnover & transaction cost analysis	52
6	Conclusion	56

1 Introduction

Over the past several decades, factor investing has become a central framework in empirical asset pricing and portfolio management. Factor investing seeks to explain and exploit systematic patterns in returns that are shared across groups of assets. Beginning with the Capital Asset Pricing Model and later extended through multifactor models such as the Fama–French three- and five-factor models, the factor literature has demonstrated that exposure to certain firm characteristics—such as size, value, profitability, and investment—helps explain differences in average returns across assets. As a result, factor-based approaches are now widely studied by academics and used by practitioners.

While much of the early factor literature focused on explaining the cross section of average returns, more recent research has emphasized that expected returns also vary predictably over time. One of the most influential findings in this area is time-series momentum: the tendency for an asset’s own past returns to predict its future returns. This insight has been shown to hold across asset classes and international markets, shifting the attention from the relative performance across assets to persistent dynamics in returns both at the individual and factor level.

Building on this idea, [Gupta and Kelly \(2019\)](#) demonstrate that momentum is not confined to individual securities but also exists at the level of systematic equity factors. Their study, *Factor Momentum Everywhere*, shows that factor returns exhibit strong serial correlation and that timing factor exposures based on their own past performance—referred to as time-series factor momentum (TSFM) – leads to substantial improvements in risk-adjusted performance relative to static factor allocations. These results suggest that factor investing can be meaningfully improved by allowing factor exposures to vary dynamically over time.

Despite the strong empirical performance of TSFM, the timing mechanism employed in the existing literature remains deliberately simple. Factor exposure decisions are based on linear functions of past factor returns, typically aggregated over fixed lookback windows and scaled by historical volatility. While this simplicity enhances interpretability and robustness, it also imposes strong restrictions on how information is incorporated into the timing signal. In particular, linear momentum rules may fail to capture nonlinearities, interaction effects, and time-varying relationships across factors—features that are increasingly emphasized in modern asset pricing research.

Recent advances in empirical asset pricing have shown that more advanced machine learning techniques are increasingly relevant. Flexible, regularized models can substantially improve return predictability by exploiting complex patterns in the data without overfitting. Studies such as [Gu et al. \(2020\)](#) demonstrate that machine learning methods outperform traditional linear models when forecasting stock returns.

These findings raise a natural question: if machine learning can improve the prediction of individual asset returns, can it also improve the timing of systematic factor returns?

This thesis addresses this question by extending the factor momentum framework of [Gupta and Kelly \(2019\)](#). Instead of relying solely on linear, rule-based momentum signals, we propose a machine-learned approach to factor timing that uses a rich set of lagged factor information to forecast future factor returns. These forecasts are then translated into dynamic factor exposures using a volatility-scaled timing rule analogous to traditional TSFM. By doing so, we aim to assess whether machine learning can extract additional predictive structure from factor return dynamics and deliver superior performance relative to existing factor momentum strategies.

2 Literature review

2.1 What is a Factor?

In asset pricing, a factor refers to a systematic source of risk or return that is shared across many assets and helps explain differences in returns. The standard approach is to construct factors as long-short portfolios based on some firm characteristic such as size, liquidity or value. Factors help capture patterns in returns that cannot be explained by idiosyncratic, firm-specific noise. Factor investing is an approach that specifically targets these systematic drivers of returns by allocating capital to portfolios with appealing factor exposure. This is in contrast to the simpler approach of selecting individual securities in isolation.

The simplest and perhaps most influential of factor models is the Capital Asset Pricing Model (CAPM) developed in the 1960s ([Sharpe, 1964](#)). The model is simple in the sense that it is a one-factor model and suggests that the market factor alone captures all priced risk. The model serves well as a basis for understanding the variation in average returns, but over the years empirical evidence has shown that the CAPM fails to explain several persistent patterns in average returns, for example the higher returns of small-cap stocks and value stocks. This led to the development of multifactor models, most notably the Fama-French 3-factor model (FF3) and the Fama-French 5-factor model (FF5).

2.2 Multifactor Models

[Fama and French \(1993\)](#) propose a 3-factor model for equity returns. In addition to the market factor, firm size and book-to-market equity are introduced as factors that capture return patterns. The authors found that by adding these factors the explanatory power of the model improves drastically over the simple CAPM model. The paper received widespread attention and is by now one of the most cited

papers in finance. Following this, there have been many contributions to asset pricing literature and factor research. [Fama and French \(2015\)](#) proposed a 5-factor model by extending their earlier 3-factor model. The additional factors were profitability and investment factors and once again the model's ability to explain the cross-section of average returns improved substantially.

How many factors are enough, and which factors should one use? In theory, there are infinitely many factors and there is not one right answer when it comes to what factors one should include. [Jensen et al. \(2023\)](#) challenge the claim that empirical asset pricing suffers from a replication crisis driven by data mining, multiple testing, and a large set of proposed factors. Instead of evaluating factors one by one using standard frequentist tests, the authors develop a Bayesian framework that jointly evaluates a large set of factors while explicitly accounting for multiple testing, correlation between factors, and economic structure.

The authors develop a global data set consisting of 153 equity factors across 93 countries. The results show that most factors do replicate, perform well out-of-sample, and remain economically meaningful once multiple testing is handled correctly. The factors are split into 13 themes which are economically interpretable and many of them contribute significantly to optimal portfolios. A key result is that the large number of factors strengthens, not weakens, the statistical evidence when they are modeled jointly.

2.3 Time-Series Momentum

The concept of time-series momentum is explored in [Moskowitz et al. \(2012\)](#). Time-series momentum refers to the tendency that assets which have experienced a positive (negative) return in the recent past continue to outperform (underperform) in subsequent periods. In other words: an asset's own past returns predict its future return. This effect is not limited to stocks — the authors show that time series momentum exists for a wide range of other asset classes such as commodities, currencies and bonds. It is also applicable across global markets. The time-series momentum factor is constructed by going long assets with positive past returns and shorting those with negative past returns. An important distinction between momentum strategies is the distinction between time-series momentum and cross-sectional momentum, the latter being relative differences in performance across assets at a point in time. In general, the paper establishes time-series momentum as a prevalent phenomenon and an important source of return across asset classes.

The factor literature focuses on explaining the cross section of returns, while time-series momentum shows that expected returns also vary over time, with assets exhibiting persistence in their own past performance. A reasonable question is whether this return predictability can be applied to the factors themselves, motivating the analysis in [Gupta and Kelly \(2019\)](#), which studies momentum in systematic

factor returns.

2.4 Factor Momentum Everywhere

The primary inspiration behind our thesis is [Gupta and Kelly \(2019\)](#).

Traditionally, momentum has been studied at the stock-level and implemented cross-sectionally by buying recent winners and selling recent losers. Although this simple strategy has produced impressive risk-adjusted returns, momentum has been considered primarily an idiosyncratic phenomenon tied to individual securities. The authors challenge this view by focusing on equity factors instead of individual securities. They analyze 65 long-short factors constructed from firm characteristics such as value, profitability, and risk measures. One of their main findings is that momentum exists at a factor level and thus is not limited to individual securities. The factor momentum strategy outperforms the simple stock momentum, industry momentum, and other commonly studied investment factors in terms of Sharpe ratio.

The authors construct 65 characteristic-based equity factors which cover the most prominent factors proposed in the asset pricing literature. Each of the factors is a large, diversified long-short portfolio. Therefore, the factors are approximately without idiosyncratic risk. The top and bottom 1% of the factors are winsorized to reduce the effect of extreme outliers and each factor is rebalanced monthly.

The statistical foundation of momentum strategies is strong serial correlation in returns. The average monthly AR(1) coefficient is 0.11 across the 65 factors. By comparison, the AR(1) coefficient for excess returns of the market is 0.07 during the sample. As demonstrated by [Moskowitz et al. \(2012\)](#), a coefficient of 0.07 is already sufficient to implement a time-series momentum strategy. 59 out of the 65 factors have a positive AR(1) coefficient and the coefficient is statistically significant for 49 of them. This is a key finding as the factor portfolios are already diversified, implying predictability in common components of returns. This suggests that momentum is a broader effect and is not limited to firm-specific noise.

Two main strategies are suggested: *time-series factor momentum* (TSFM) and *cross-section factor momentum* (CSFM).

2.4.1 TSFM

For every factor, exposure is scaled based on the factor's own recent performance. If the factor exhibits positive returns in the recent past, it is held long, otherwise the factor is held short. Multiple formation periods are considered, ranging from one month to five years. The results show that timing the factors

based on their own past returns yields significant alpha relative to a static investment strategy where the factors are not timed. The factor strategies are then aggregated into an ultimate TSFM portfolio which improves the performance even further. This portfolio achieves a Sharpe ratio of 0.84 in the sample, by far exceeding that of traditional stock momentum, industry momentum and other common equity factor strategies. An important feature of TSFM is its resilience to different formation periods. Traditional stock momentum strategies typically exhibit short-term and long-term reversals. This is not the case for TSFM; it performs well both at very short (one month) and very long (five years) look-back windows.

2.4.2 CSFM

In the second approach, factors are ranked relative to one another based on recent performance. The strategies are very similar in terms of performance but differ slightly in the way they are constructed. For example, if all factors have positive recent returns, TSFM would long all of them. On the other hand, CSFM would long only the factors that relatively outperformed the others and short those with below-median returns — despite their returns being positive. The two strategies are highly correlated with a correlation coefficient above 0.9 for any formation window. The average returns and Sharpe ratios are also very similar, but when regressing TSFM returns on CSFM returns, there is a highly positive and statistically significant alpha. When performing the opposite regression, the alpha is generally negative (and statistically significant). Their high correlation but opposite sign alphas suggests that TSFM and CSFM are fundamentally the same phenomenon, but TSFM captures factor momentum in a more efficient manner than CSFM.

2.4.3 Other Results

Another important result of the paper is how factor momentum compares to traditional momentum strategies. TSFM explains a large share of the returns to stock momentum (UMD) and industry momentum, especially at intermediate horizons. The opposite does not hold true; TSFM cannot be explained by traditional momentum strategies. Additionally, a common drawback of traditional momentum strategies is that they tend to perform poorly during times of financial distress. This is not the case with factor momentum, making it appealing from a portfolio construction perspective.

The authors find that factor momentum is complementary to both stock momentum and value strategies. When included in mean-variance efficient portfolios alongside traditional factors, factor momentum is given significant positive weight and improves Sharpe ratios. These types of strategies have high turnover by construction, but even after accounting for transaction costs the performance remains strong.

The paper further documents that factor momentum is a global phenomenon. When using international equity data (excluding the United States), a similar persistence in both factor returns and performance is found. Factor momentum still outperforms stock and industry momentum and is robust across formation periods.

[Gupta and Kelly \(2019\)](#) make several important contributions to the momentum literature. Most importantly, they show that momentum is not limited to individual stocks but also appears at the level of systematic equity factors. Their analysis suggests that factor momentum is more effectively implemented using a time-series approach rather than a purely cross-sectional comparison across factors. Furthermore, they demonstrate that factor momentum explains a large share of the returns typically associated with stock-level momentum, while at the same time delivering more stable performance and stronger diversification benefits. Overall, the paper broadens the understanding of momentum by framing it as a general property of expected returns that operates across different levels of aggregation. Thus, it provides strong empirical support for including factor momentum as a central component in multifactor investment strategies.

The main takeaway which is important for our thesis is that factors can be timed to improve the performance of factor investing.

2.5 Machine Learning & Asset Pricing

The use of machine learning in financial modeling has become more popular in recent years. The traditional linear models fail to capture the non-linearity of financial returns in modern markets. With the increase in available data and predictors, the need for tools that can handle high-dimensional and complex data arises. With machine learning techniques, one can identify complex interaction effects and structural dependencies that linear models simply cannot capture. Several papers in recent years highlight the benefits of machine learning in finance. We will briefly discuss the main findings of the papers on this topic that are the most relevant for our thesis.

2.5.1 The Virtue of Complexity in Return Prediction

[Kelly et al. \(2024\)](#) challenge the “simple” return prediction models which are widely used in finance. The authors examine whether complex models – models where the number of features (P) exceeds the number of observations (T) – can better predict returns and enhance portfolio performance relative to “simple” models. Contrary to standard economic theory, the authors prove that expected out-of-sample forecast accuracy and portfolio Sharpe ratio are strictly increasing in model complexity when the appropriate shrinkage is applied. They argue that “overparameterized” models can in fact be a virtue

when the true data generating process (DGP) is unknown because the statistical costs of a complex model are outweighed by the approximation increase achieved when using a model with many parameters. In a high-complexity environment ($P > T$), models produce accurate out-of-sample forecasts even when the training data is perfectly fitted with zero error. Furthermore, they find that R^2 is an insufficient measure of a model's economic value. A timing strategy can still be profitable and have a positive Sharpe ratio even when R^2 is negative. The reason is that R^2 is highly sensitive to forecast variance while the direction of the prediction can still be correct. By using a sample of U.S. equity market returns from 1926 to 2020 and standard predictors, complex models outperform the simple linear models significantly with Sharpe ratio increasing 0.47 p.a. over a simple buy-and-hold strategy.

2.5.2 Empirical Asset Pricing via Machine Learning

[Gu et al. \(2020\)](#) compare different machine learning methods in the context of asset pricing, more specifically measuring the asset risk premiums. A large sample of U.S. stocks over a 60-year period is analyzed using over 900 signals, spanning stock-level, macroeconomic, and sector-level variables. Several methods are compared, such as simple and penalized regressions, various dimension reduction techniques, boosting, random forests and neural networks. The main finding of the paper is that machine learning has the potential to improve the empirical understanding of asset returns. Neural networks and trees show the best performance with R^2 values that exceed traditional regressions. The outperformance of these can be traced back to their ability to capture nonlinear interactions that linear models are unable to. The statistical improvement of these methods can translate into considerable economic gains for investors. By using neural network forecasts, timing the S&P-500 achieves an out-of-sample Sharpe ratio of 0.77 compared to 0.51 for the buy-and-hold strategy. Common for all the methods is that some variables are more informative when predicting returns. The most informative predictors can be categorized into price trends (momentum and reversal), liquidity and volatility. Furthermore, the authors document that shallow learning outperforms deep learning in their analysis. The performance of neural networks peaked at three hidden layers and trees with few leaves were often selected. The outperformance of shallow learning is likely linked to the low signal-to-noise ratio exhibited by financial returns. Finally, the patterns in S&P-500 forecast performance also carry over to factor portfolios.

2.5.3 Nonlinear Time Series Momentum

[Moskowitz et al. \(2025\)](#) explore the nonlinear relationships between price trends and assets' future returns across global markets and asset classes. Conventional time series momentum (TSMOM) strategies usually rely on simple binary functions or linear weights. The authors instead suggest a nonlinear weighting scheme, using machine learning and neural networks. They find that nonlinear momentum is

a pervasive phenomenon across both asset classes, horizons and frequencies. More importantly, they show that nonlinear time series momentum (NLTS MOM) outperforms other TSMOM weighting schemes consistently across both lookback periods and frequencies. The most interesting part is that these results also hold in the context of long-short equity factor momentum, directly relating to our thesis and the hypothesis that machine learning techniques can further enhance the performance of time series factor momentum. The key difference here, though, is that the authors focus on predicting the weights rather than predicting the actual returns, which will be our primary objective.

We believe that incorporating machine learning into our analysis could be beneficial and has the potential to improve on the already impressive results shown by [Gupta and Kelly \(2019\)](#). Specifically, our hypothesis is that by exploiting the benefits of machine learning algorithms—most notably their ability to handle nonlinearities and interaction effects—we can further improve on the timing mechanism in the same way that [Gu et al. \(2020\)](#) and [Kelly et al. \(2024\)](#) have shown that machine learning improves the return prediction of stocks. Instead of focusing on return predictability in stocks, we shift our attention to return predictability in factors.

3 Research methodology

Our implementation contains three main parts: replication of the main results in [Gupta and Kelly \(2019\)](#), implementation of the machine learning strategy, and comparison of results.

3.1 Use of AI Disclosure

Throughout our thesis, we have made use of Claude (Anthropic) and ChatGPT (OpenAI) as AI-assisted tools. Specifically, we have used AI to support code development and debugging in Python and LaTeX, as well as assisting with language and phrasing in certain parts of the text. All content, arguments, interpretations, and conclusions presented in this thesis are our own, and we take full responsibility for the accuracy and integrity of everything. The use of AI tools did not substitute for our own judgement, and all AI-assisted outputs were critically reviewed by us before inclusion.

3.2 Implementing TSFM

The empirical analysis is based on stock-level data obtained from the JKP database through WRDS. The JKP database provides standardized firm characteristics and return data constructed from underlying CRSP and Compustat inputs. We use these stock-level observations as the input to our own factor construction. Specifically, for each characteristic we sort stocks into portfolios using the procedure described below, and then construct monthly long-short factor returns closely following the methodology

of [Gupta and Kelly \(2019\)](#). The factors are clustered into 13 economic themes. Examples of factors include:

- Valuation ratios (earnings-to-price, dividend yield, book-to-market equity)
- Risk measures (beta, idiosyncratic volatility)
- Profitability measures (profit margin, return on equity)
- Size measures (market capitalization)

The factors are constructed as outlined by [Gupta and Kelly \(2019\)](#). Stocks are sorted based on the underlying characteristic after winsorizing the top and bottom 1% of the raw characteristic values each period. Then, the cross-section is split by size (large vs. small) and is further divided into low/medium/high characteristic values according to a 30/40/30 percentile split within each size bin. The result is six size bins and value-weighted portfolios are constructed according to $0.5 \cdot (\text{Large High} + \text{Small High}) - 0.5 \cdot (\text{Large Low} + \text{Small Low})$, with portfolios rebalanced each month. In our thesis we follow the same methodology when constructing the factors.

We will implement time-series factor momentum in a similar fashion to the procedure for TSFM described in *Factor Momentum Everywhere*, but as mentioned we will be using machine learning which adds another layer of complexity to the analysis. The original paper suggests a weighting scheme where factor returns over the past j months are scaled by the annualized volatility of the factor. The exposure to each factor in the original paper is determined by a volatility-scaled momentum signal that is capped at ± 2 in each period, which indirectly bounds factor exposures and limits the influence of extreme signals. Mathematically, it is given by:

$$s_{i,j,t} = \min \left(\max \left(\frac{1}{\sigma_{i,j,t}} \sum_{\tau=1}^j f_{i,t-\tau+1}, -2 \right), 2 \right) \quad (1)$$

and the factor return next period is:

$$f_{i,j,t+1}^{\text{TSFM}} = s_{i,j,t} \times f_{i,t+1} \quad (2)$$

To implement the overall TSFM strategy we again do the same as described in [Gupta and Kelly \(2019\)](#). The aggregated TSFM portfolio is constructed as:

$$\text{TSFM}_{j,t} = \text{TSFM}_{j,t}^{\text{Long}} - \text{TSFM}_{j,t}^{\text{Short}} \quad (3)$$

where

$$\text{TSM}_{j,t}^{\text{Long}} = \frac{\sum_i \mathbf{1}_{\{s_{i,j,t} > 0\}} f_{i,j,t}^{\text{TSM}}}{\sum_i \mathbf{1}_{\{s_{i,j,t} > 0\}} s_{i,j,t}} \quad (4)$$

and

$$\text{TSM}_{j,t}^{\text{Short}} = \frac{\sum_i \mathbf{1}_{\{s_{i,j,t} \leq 0\}} f_{i,j,t}^{\text{TSM}}}{\sum_i \mathbf{1}_{\{s_{i,j,t} \leq 0\}} s_{i,j,t}} \quad (5)$$

Note that we only consider $j = 1$.

3.3 Alternative implementation of aggregated TSM

In the implementation of the aggregated TSM strategy described in [Gupta and Kelly \(2019\)](#) one might notice that the long and short sides are weighted separately, and then combined into a portfolio after separate creation. While this method has its merits, we believe another version of it may also be interesting to look at. One potential disadvantage of the traditional TSM portfolio is that it does not allow for a net bias in being long/short factors, which may be appropriate if the majority of predictions are positive or negative. Dollar-neutrality still holds if one has a net bias on factors due to the fact that the underlying factors are dollar-neutral. This leads us to come up with an alternative version of TSM which we will call pooled TSM, we define it below¹.

$$s_{i,t} = \min \left(\max \left(\frac{f_{i,t}}{\sigma_{i,t}}, -2 \right), 2 \right) \quad (6)$$

$$w_{i,t} = \frac{s_{i,t}}{\sum_{j=1}^N |s_{j,t}|} \quad (7)$$

$$\text{Pooled TSM}_t = \sum_{i=1}^N w_{i,t} f_{i,t+1} \quad (8)$$

We call it pooled TSM since we pool all the weights in the long and short legs together. Another notable difference in this formulation is that gross exposure to factors will always be 1 compared to 2 for regular TSM. Net exposure to factors can vary from -1 to 1.

3.4 Machine-Learned Time-Series Factor Momentum

The methodology we use to form MLTSM (Machine-Learned Time-Series Factor Momentum) portfolios is similar to how [Gupta and Kelly \(2019\)](#) form TSM portfolios, with some small but key differences. We only focus on one-month formation periods as signals to our model. This is done for several reasons,

¹Note that since we only consider $j = 1$ formation periods we ignore the formation period subscript in these equations.

partly to accommodate our added machine-learning dimension rather than formation period dimension, but also due to the computational resource constraints. We note that having several formation periods as signals to our model may be of value, but we do not have the resources to bear the known computational cost of proper training. Apart from this, we will also be scaling the one-month ahead prediction of the i th factor similarly to what is done for TSFM:

$$f_{i,t+1}^{MLTSFM} = s_{i,t} \times f_{i,t+1} \quad (9)$$

$$s_{i,t} = \min \left(\max \left(\frac{1}{\sigma_{i,t}} \mathbb{E} [f_{i,t+1} \mid \Omega_t], -2 \right), 2 \right) \quad (10)$$

Our scaling factor is similar to that of [Gupta and Kelly \(2019\)](#), but instead of using historical factor returns we use the one-month ahead predicted returns (according to our model), given all information we have at time t , to decide if we are going long or short the factor. We then divide this by the monthly standard deviation of factor returns over the previous three years to obtain a z-score, which helps in determining appropriate leverage, and is capped at ± 2 . Note that we do not expect this score to be distributed as anything close to a standard normal since we are dividing by the standard deviation of actual returns.

The overall MLTSFM portfolio is determined in a similar way to TSFM, but we must take care of an important edge case that otherwise would lead to the return being undefined in certain periods. The volatility of predicted returns is much lower than the volatility of actual returns; with the mean of predicted returns also being positive this leads to a few periods where all of our return predictions are positive, resulting in a short leg without any factors. To address this we simply define the short leg return to be 0 in periods where no negative predictions exist. In these periods it results in a long-only factor strategy that is not factor neutral². The pooled MLTSFM portfolio is determined in the same way as pooled TSFM, the only difference being a replacement of actual factor return with predicted factor return when calculating $s_{i,t}$.

The most important part of this process is determining $\mathbb{E} [f_{i,t+1} \mid \Omega_t]$, our prediction for the one-month ahead factor return. We approach this problem by selecting a class of models that adequately handles three key challenges, two of which are outlined by [Gu et al. \(2020\)](#), and one we identify ourselves.

The first challenge is that the number of variables we can condition the expected return on is very large. The concept of the "factor zoo" coined by [Cochrane \(2011\)](#) points towards the ever growing list of po-

²Even though the aggregate MLTSFM portfolio is not 1 dollar long and 1 dollar short a multi-factor portfolio in such a case, the strategy is still dollar-neutral as the underlying portfolios are zero-cost portfolios.

tential return explaining factors. Due to this, we need a model that can handle high dimensional data, and determine which features are truly important for the prediction problem.

Second, we do not know the exact relationship between our feature set and our label, that is the functional form that the data-generating process of return takes. This could be highly non-linear with many interaction effects between our features. So, we need a model that can handle potential non-linearities and interaction effects on its own, without us needing to explicitly define new features that incorporate this.

Third, for each of our factors we will need to retrain the model as time passes and we get new information, this results in potentially thousands of models needing to be trained per factor depending on how one searches for optimal hyperparameters. It is therefore essential that the model and the algorithm used to implement the model are computationally efficient.

Gradient boosted trees handle all these challenges well, and therefore we select this class of models for our analysis. We compare three popular implementations of gradient boosting: XGBoost (eXtreme Gradient Boosting), LightGBM (Light Gradient Boosting) and CatBoost (Categorical Boosting).

3.4.1 Regression Trees & Gradient Boosting

To understand why we choose to use gradient boosted regression trees in our analysis we must first look at what regression trees and gradient boosting are separately.

Decision trees (or regression trees for a regression task) attempt to divide the feature space into non-overlapping regions, then the prediction is taken to be the mean value (for an MSE loss) of the label within each of the regions. A single regression tree as seen in Figure 1 is a very interpretable model, where a data point is sent either left or right at each split depending on what value it takes, this continues until we reach the terminal nodes where a prediction of the label is made.

By allowing each split to be on a different feature, regression trees can model interaction effects between features. If we were to plot the prediction of a regression tree versus one feature (for an arbitrary problem) then the resulting function would quite often resemble a step function, which shows that regression trees can handle non-linearities. Regression trees also perform feature selection, each split is only a split on a single feature, it will therefore naturally only select features that lead to the largest performance gain.

Even though regression trees have some quite attractive properties with respect to what is desired for this task, single regression trees on their own tend to produce poor forecasting results. This is potentially

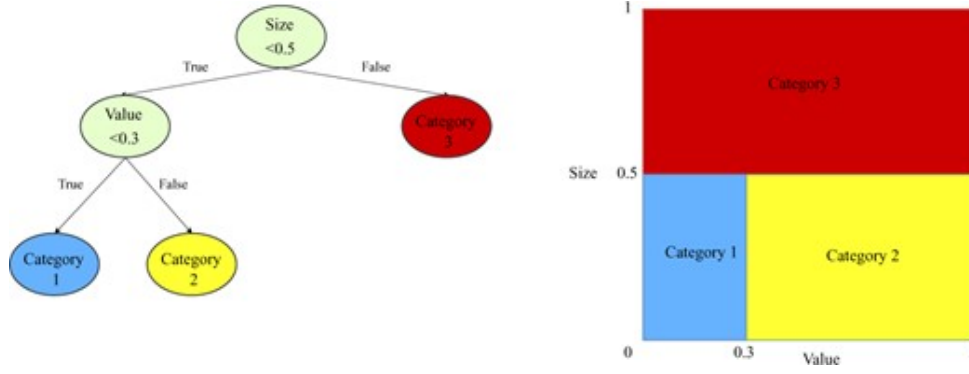


Figure 1: Regression tree example from [Gu et al. \(2020\)](#)

due to them being quite challenging to optimize the bias-variance tradeoff with, leading to an underfit or overfit. In other words, deep trees (many splits, many terminal nodes) are needed to properly capture complex functions, but deep trees are also highly unstable.

To address the shortcomings of regression trees we can work with gradient boosted trees. Gradient boosting is an ensemble model technique, where the goal is to fit many weak learners iteratively to create a strong learner. Often the weak learners (weak models) we fit are shallow regression trees. The idea behind this is that we can increase the stability of our overall model by averaging out our prediction over many trees, whilst still keeping the benefits of regression trees. XGBoost, LightGBM and CatBoost are all well-established implementations of gradient boosting and are widely used in both academic research and machine learning prediction competitions.

3.4.2 Hyperparameters

Frameworks that use gradient boosted trees have different hyperparameters that must be defined before training. Unlike ordinary model parameters, which are learned from the data during training, hyperparameters direct the behavior of the learning algorithm itself and thus need to be specified in advance. Some of the most common hyperparameters in gradient boosting are the number of trees M , the maximum depth of each individual tree d , the regularization parameter λ , and learning rate η . The number of trees determines how many weak learners are combined, while the maximum depth restricts the complexity of each individual tree and as a result influences the bias-variance trade-off ([Hastie et al., 2009](#)). The regularization parameter λ reduces overfitting by penalizing more complex models ([Chen and Guestrin, 2016](#)), and learning rate η scales the contribution of each new tree to the ensemble.

We will briefly discuss some additional hyperparameters that are relevant for our analysis. XGBoost and LightGBM have `colsample_bytree`, a parameter that controls the fraction of features that can be used to grow a tree. When set to 1, all features are available. When set to 0.5, only 50% of the

features are available and so on. The purpose of setting `colsample_bytree` to anything less than 1 is to reduce correlation between trees and prevent overfitting. The logic is similar to what is done for random forests ([Breiman, 2001](#)), but done in a boosting framework instead of a bagging framework. Taking the average over many different trees vs. many similar trees can be beneficial as it tends to result in a better final prediction ([Hastie et al., 2009](#)). The equivalent of `colsample_bytree` for CatBoost is `rsm` and serves the same purpose ([Prokhorenkova et al., 2018](#)). `min_child_weight` sets a minimum weight required to form another leaf ([Chen and Guestrin, 2016](#)). If a split results in a leaf containing less weight than `min_child_weight`, the split is not performed. Essentially, this parameter ensures that further splitting needs to be informative enough to justify the split. The CatBoost semi-equivalent is `min_data_in_leaf` but instead of setting a minimum weight a minimum number of observations is set. Finally, LightGBM uses `num_leaves` which controls the maximum number of leaves per tree instead of depth ([Ke et al., 2017](#)).

3.4.3 Bayesian Optimization & Hyperparameter Tuning

We have several options when it comes to hyperparameter tuning. The most common methods used are grid search, random search and Bayesian optimization. When performing a grid search, one begins by specifying the set of hyperparameters to tune, such as tree depth and learning rate, along with a grid of candidate values ([Bergstra and Bengio, 2012](#)). Grid search goes through every single combination of hyperparameters within the specified grid. The advantage of this method is that it is, in theory, guaranteed to find the best possible combination of hyperparameter values in the grid, given that cross-validation accurately estimates the test-error. The drawback is that it is computationally expensive and quite slow, especially with many hyperparameters and possible hyperparameter values. In contrast to grid search, random search samples combinations of hyperparameter values from the defined search space at random ([Bergstra and Bengio, 2012](#)). It is considerably less computationally expensive than grid search and a lot faster. Random search can often identify a good solution with relatively few iterations, but unlike grid search, it is not guaranteed to find the best-performing combination within the search space.

The third option is Bayesian optimization. We will briefly introduce the concept, but we encourage the reader unfamiliar with the topic to check out additional resources. Instead of testing all possible combinations or sampling at random, Bayesian optimization takes a different approach ([Snoek et al., 2012](#)). The key difference is that Bayesian Optimization learns from past iterations to decide where in the hyperparameter space to search next. It is therefore considerably more efficient than both grid search and random sampling. The method implements a surrogate model, usually a Gaussian Process ([Rasmussen and Williams, 2006](#)), which is used as a probabilistic approximation of the objective function across the

hyperparameter space. The surrogate model is updated after each evaluation, and gradually identifies which regions of the search space are the most likely to perform well. An acquisition function is used to determine where to evaluate next. Two commonly used acquisition functions are expected improvement (EI) and probability of improvement (PI). They both attempt to balance two objectives: sampling in already promising areas (exploitation) and sampling in uncertain areas where better solutions can exist (exploration) ([Shahriari et al., 2016](#)). This trade-off enables Bayesian optimization to converge to a solution which is near optimal in far fewer iterations than grid search or random search. For these reasons, Bayesian optimization is our preferred choice for our thesis.

3.4.4 ML model training regime

The use of machine learning introduces a set of additional methodological challenges that must be addressed. First, machine learning algorithms require a sufficiently long training sample to produce reliable forecasts. Since the data we are using are observed at a monthly frequency, a minimum estimation window of approximately 20 years (240 observations) is imposed to ensure adequate model performance. This effectively reduces the available sample size relative to that used in [Gupta and Kelly \(2019\)](#), particularly in earlier periods of the data. As a result, there is a trade-off between model accuracy and the length of the usable time series for analysis.

A second important consideration concerns the frequency with which the model should be retrained. In principle, re-estimating the model every month would allow it to incorporate the most recent information and adapt quickly to changes. However, this approach is quite computationally intensive, and retraining a separate model each month for every factor may potentially offer limited incremental benefits relative to its computational complexity. As a result, we will explore less frequent retraining schemes and focus on evaluating the stability of the learned relationships over time, rather than relying solely on high-frequency model updates.

The machine learning problem we are faced with is a time-series prediction problem. To create the MLTSFM portfolio we perform the following process for each factor:

1. Define the feature matrix as the set of all available raw factor returns lagged by one month. For each prediction task, the label is the one-month-ahead return of the factor being predicted. Thus, each model uses lagged information from the full factor universe to forecast the next-month return of a single target factor. Factor returns available are a subset of factors studied in [Gupta and Kelly \(2019\)](#).
2. Set the minimum training period to 20 years. For example, if the data starts in 1965, the first

forecast of

$$\mathbb{E} [f_{i,t+1} \mid \Omega_t]$$

will be in 1985. We consider both an expanding training data set and a fixed-size training data set where the fixed size is 20 years.

3. Perform hyperparameter search using Bayesian optimization (25 iterations) with time-series cross-validation as the evaluation strategy. The hyperparameters tuned are: M (number of trees), d (maximum depth of an individual tree), λ (L2 penalization parameter), and a set of implementation-specific parameters. We use a learning rate $\eta = 0.01$ for all boosting implementations.
4. Train the model on the full training data set using the optimal hyperparameters.
5. Predict factor returns for the next five years.
6. Move the training origin five years forward, updating the expanding or fixed training window accordingly. Repeat steps 2–6 until reaching the end of the data set.

After computing predictions for all available factors we construct the MLTSFM portfolio for individual factors and the overall portfolio as described in subsection 3.4.

3.5 Performance comparison of results

Performance in Factor Momentum Everywhere is evaluated using a combination of average realized returns, Sharpe ratios, and regression-based alpha measures to assess both absolute and risk-adjusted performance. An important comparison is between dynamic factor timing strategies and static buy-and-hold factor portfolios. Average realized returns and Sharpe ratios are used to measure economic performance. This allows the authors to compare factor momentum strategies directly with other investment strategies such as traditional factors.

In addition, the authors estimate alphas from time-series regressions in which returns to factor momentum strategies are benchmarked against the underlying static factor returns. These regressions isolate the value that is added by the timing element—separating performance due to dynamic exposure from compensation for holding the factor itself. A strategy adds value if it delivers a positive and statistically significant alpha relative to the benchmark of a static factor. This approach ensures that performance gains reflect genuine timing ability rather than mechanical exposure to known risk premia. We will follow the same procedure in our thesis when evaluating the MLTSFM portfolio. In addition to these measures, we need to compare the performance of the MLTSFM portfolio to the TSFM portfolio described in [Gupta and Kelly \(2019\)](#).

For all regression-based estimates we report standard errors computed using the Newey-West heteroskedasticity and autocorrelation consistent (HAC) estimator. This is especially important if the monthly factor returns are serially correlated, which would otherwise lead to understated standard errors and overstated statistical significance under standard OLS assumptions (Newey and West, 1987). The number of lags is chosen according to the rule of thumb $L = \lfloor 4(T/100)^{2/9} \rfloor$, where T is the number of observations. Confidence intervals reported throughout Section 5 are constructed as the point estimate ± 1.96 times the Newey-West standard error.

Throughout this thesis, all reported returns are *excess returns*, i.e. returns in excess of the risk-free rate. Since the factors we are studying are long-short (zero cost), the risk-free rate is removed in the return calculation. Therefore, focusing on excess returns is the natural choice.

4 Data collection

We obtain stock-level return and characteristic data from the JKP database through WRDS. The JKP data are constructed from underlying CRSP and Compustat inputs, and we use these observations to construct the factor portfolios used throughout the analysis. Since the data are obtained from established academic and financial databases through WRDS, the analysis does not involve personal or sensitive data. All empirical work is conducted in Python, using libraries such as pandas, NumPy, scikit-learn, XGBoost, LightGBM, and CatBoost.

4.1 Differences in Implementation

We will attempt to follow the methodology described in Gupta and Kelly (2019) as closely as possible. However, sometimes it is necessary to deviate a little due to either data limitations or practical reasons.

We restrict our analysis to US stocks only, whereas Gupta and Kelly (2019) also analyze the international sample. This choice is motivated by several practical considerations. First, the US market has the longest and most complete data sample, which is important given our requirement of a minimum 20-year training period before generating out-of-sample predictions. Second, by focusing on US stocks we avoid potential complications arising from currency risk, differences in accounting standards, and different risk-free rates across countries.

We have had some challenges with collecting data for all of the characteristics described in Gupta and Kelly (2019) when using the JKP data source. For some of the missing characteristics it seems that the data is simply not available in the JKP database, whilst for others we are not completely confident as to what characteristic is being referred to. As mentioned, we need an initial training period before we can

make predictions of factor returns. With a minimum requirement of 20 years of data for training, the first predictions begin in 1985. For two of the factors the data begins after 1985, therefore we exclude these.

A critical deviation from [Gupta and Kelly \(2019\)](#) concerns the construction of the betting-against-beta (BAB) factor. All the factors are constructed as value-weighted long–short portfolios based on 30/40/30 percentile breakpoints and NYSE median market capitalization, combined as $0.5 \times (\text{Large High} + \text{Small High}) - 0.5 \times (\text{Large Low} + \text{Small Low})$. Although this approach is appropriate for most factors, it is theoretically inconsistent with the BAB factor. As originally proposed by [Frazzini and Pedersen \(2014\)](#), BAB is specifically designed to be beta-neutral, with each leg leveraged or deleveraged to a beta of one. Applying the standard value-weighted long-short construction to BAB ignores this step, resulting in a portfolio that has residual market exposure. We did initially create the factor following the same framework as the other factors, but the resulting factor did not behave as expected. It is entirely possible that [Gupta and Kelly \(2019\)](#) have constructed the factor correctly. After all, it is one of their best performing factors in regard to its performance metrics. Still, it is not explicitly stated that this factor is constructed differently from the others. As there is uncertainty around the exact construction process, we have decided to leave it out entirely. The final list consists of 48 characteristics that we will use further in our analysis.

5 Results & Analysis

5.1 Overview of Factors

Using the 48 characteristics we have constructed factor portfolios, where the average return and Sharpe ratio as well as the FF5-alpha is reported in [Table 1](#).

Table 1: Average Returns, Sharpe Ratios, and Alphas

Factor	$\bar{R}_{\text{ann}}$	$\text{Sharpe}_{\text{ann}}$	α_{FF5}
ami_126d	-0.0128	-0.2109	0.0050
at_gr1	0.0265	0.3234	-0.0000
at_turnover	0.0444	0.5947	0.0030
be_me	0.0161	0.1215	0.0000
bev_mev	0.0044	0.0353	-0.0189
capex_abn	0.0334	0.6941	0.0331
capx_gr2	0.0386	0.5868	0.0273
cash_at	0.0176	0.1659	0.0552
cowc_gr1a	0.0347	0.7875	0.0517
debt_me	0.0072	0.0550	-0.0390

Continued on next page

Table 1 – continued

Factor	$\bar{R}_{ann}$	Sharpe _{ann}	α_{FF5}
div12m_me	0.0083	0.0584	-0.0248
ebit_sale	0.0270	0.2604	-0.0047
ebitda_mev	0.0563	0.4642	0.0069
f_score	0.0276	0.3600	0.0011
fcf_be	0.0895	1.0174	0.0497
fcf_me	0.0909	0.7790	0.0258
gp_at	0.0542	0.6918	0.0326
inv_gr1	0.0384	0.6201	0.0322
iskew_ff3_21d	-0.0013	-0.0320	-0.0068
ivol_ff3_21d	0.0411	0.2669	0.0098
lnoa_gr1a	0.0428	0.6277	0.0190
market_equity	-0.0090	-0.1408	0.0040
ncoa_gr1a	0.0453	0.6735	0.0260
netdebt_me	0.0051	0.0411	0.0546
ni_be	0.0490	0.4614	-0.0031
ni_me	0.0415	0.3057	-0.0078
noa_at	0.0591	0.9076	0.0693
o_score	0.0412	0.5224	0.0155
oaccruals_at	0.0502	0.7438	0.0684
ope_be	0.0671	0.6830	-0.0000
opex_at	0.0422	0.6056	0.0127
ppeinv_gr1a	0.0364	0.4834	0.0181
qmj_prof	0.0694	0.8238	0.0327
rd5_at	0.0227	0.1788	0.0673
rd_me	0.0451	0.3968	0.0756
rd_sale	0.0122	0.0879	0.0725
ret_12_1	0.0657	0.4232	0.0590
ret_1_0	0.0271	0.2288	0.0234
ret_60_12	0.0124	0.1296	-0.0025
rmax1_21d	0.0437	0.2881	0.0171
sale_bev	0.0611	0.9066	0.0422
sale_gr3	-0.0031	-0.0401	-0.0126
sale_me	0.0414	0.3314	-0.0216
saleq_su	0.0104	0.1668	0.0169
seas_2_5an	0.0402	0.6045	0.0457
tangibility	0.0216	0.3124	0.0391
turnover_126d	-0.0029	-0.0191	-0.0102
z_score	0.0121	0.1347	0.0228

A good-performing factor should ideally score high in all three metrics. We can see from the table that factors with good scores in one metric usually have high scores in the other two metrics as well. Though, this is merely an observation and is not always the case.

The best performing factor in [Gupta and Kelly \(2019\)](#) in terms of Sharpe ratio is BAB with a Sharpe ratio of 0.9. This factor also has the highest average return with 9.7% and is 5th best in terms of FF5 alpha. For reasons previously discussed, we have decided to leave out this factor entirely from our analysis. We have no equivalent factor to BAB so making any direct comparison here is unfeasible. However, it is important to note that leaving this out impacts the further analysis. For instance, when we later construct the TSFM portfolio, BAB gets no weight as it is not included in our factor universe. Other top-performers are stock momentum, industry momentum and changes in non-current operating assets, among others.

When comparing our results to that of [Gupta and Kelly \(2019\)](#) it is important to keep in mind that instead of their 65 factors we have 48. Additionally, our sample period is 1985 – 2024 while theirs is 1960s – 2019. Some deviations in results are therefore expected. Our best performer in terms of Sharpe ratio is free cash flow to book equity (`fcf_be`) with an impressive 1.02, considerably higher than the already solid 0.63 [Gupta and Kelly \(2019\)](#) report. The alpha is also notably higher at 4.97% vs. 3.2%. Other factors worth noticing are `noa_at`, `sale_bev` and `qmj_prof` with Sharpe ratios of, respectively, 0.91, 0.91 and 0.82. One of the biggest differences is `sale_bev`, defined as sales to book enterprise value. The equivalent in [Gupta and Kelly \(2019\)](#) is SP (sales to price). The factors are related, though not completely identical which may explain some of the discrepancy. The average return, Sharpe ratio and FF5 alpha are 6.11%, 0.91 and 4.22% compared to 4.9%, 0.55 and -0.4% for its counterpart SP. The fact that they are slightly different is true for several other factors as well, and is something that must be kept in mind when comparing the results. Overall, some findings are quite similar to those in [Gupta and Kelly \(2019\)](#), while others deviate substantially.

A natural benchmark for comparison is the FF5 factor portfolios. Most of the factors exhibit positive and statistically significant alphas, suggesting that the benchmark does not fully capture the risk premia present in these characteristics. The factors `be_me`, `at_gr1` and `ope_be` all have an alpha of exactly zero. This is expected as it is a result of model construction rather than an empirical finding. These factors are identical to the FF5 factors used in the regressions. In a linear regression, a variable regressed on itself will yield an alpha of zero and a beta of one by construction. What is interesting is that [Gupta and Kelly \(2019\)](#) do not report alphas of exactly zero for these same factors. The most likely explanation is that the discrepancy stems from small differences in portfolio construction methodologies. While we do not consider this discrepancy a major issue, it is odd that the original paper did not find the same results.

We have followed the factor construction methodology described by French (2026) on his official data library website. Figure 2 provides a visual representation of the results in table 1 along with confidence intervals.

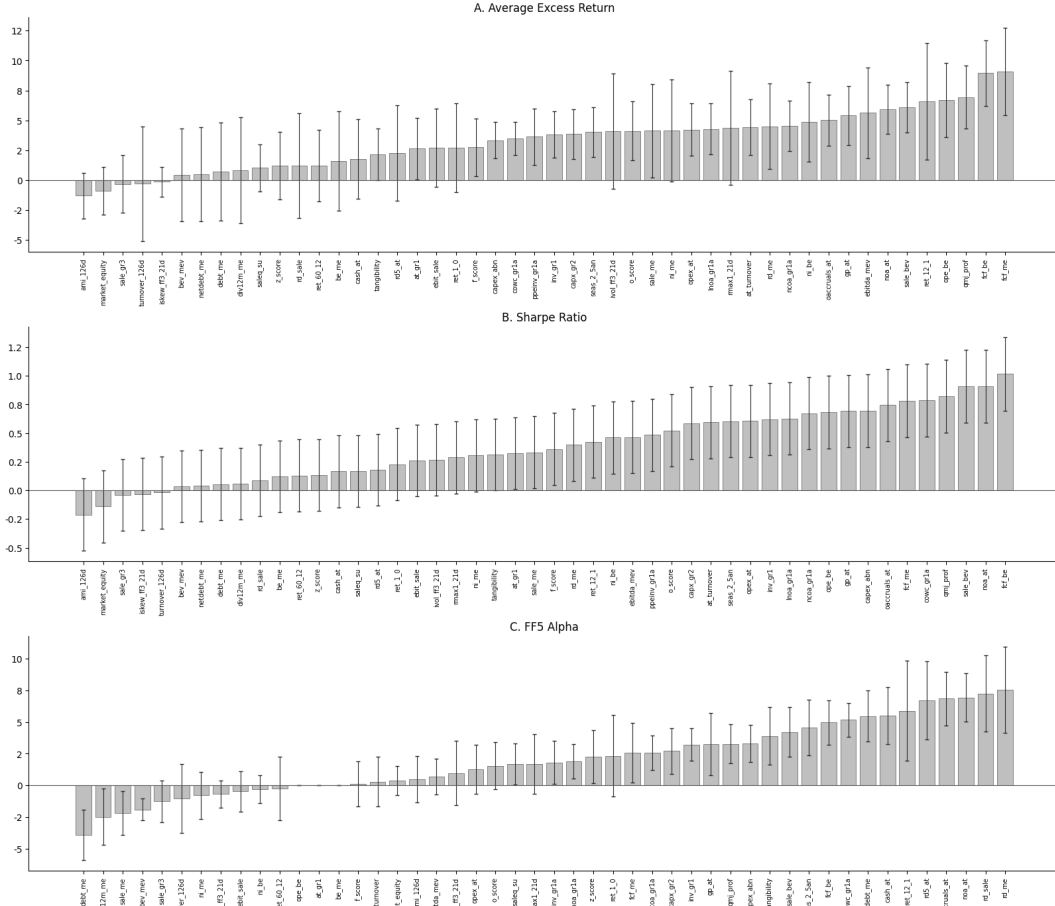


Figure 2: Average Excess Returns, Sharpe Ratios, and FF5 Alphas for Raw Factors. Panel A shows annualized average excess returns, Panel B shows annualized Sharpe ratios, and Panel C shows annualized FF5 alphas. Error bars represent 95% confidence intervals.

5.2 Autocorrelation in Factor Returns

Before evaluating the performance of timing strategies, it is useful to first establish whether factor returns exhibit any predictable patterns that a timing strategy could exploit. Figure 3 presents the AR(1) coefficients and 95% confidence intervals for all of our factors. We calculate the monthly first-order autocorrelation coefficient (AR(1) coefficient) for each factor's return series just as Gupta and Kelly (2019) do. A coefficient which is positive and significant suggests that the factor returns exhibit momentum. Out of the 48 factors, 44 (91.7%) show a positive AR(1) coefficient, of which 31 (64.6%) are significant at the 95% confidence level. None of the negative AR(1)-coefficients are statistically significant. These results are close to what Gupta and Kelly (2019) found; 59 out of their 65 factors (90.8%) had a positive coefficient and 49 (75.4%) were statistically significant. Note that we use Newey-West standard errors;

when using OLS standard errors, the number of significant positive coefficients increases to 36 (75%).

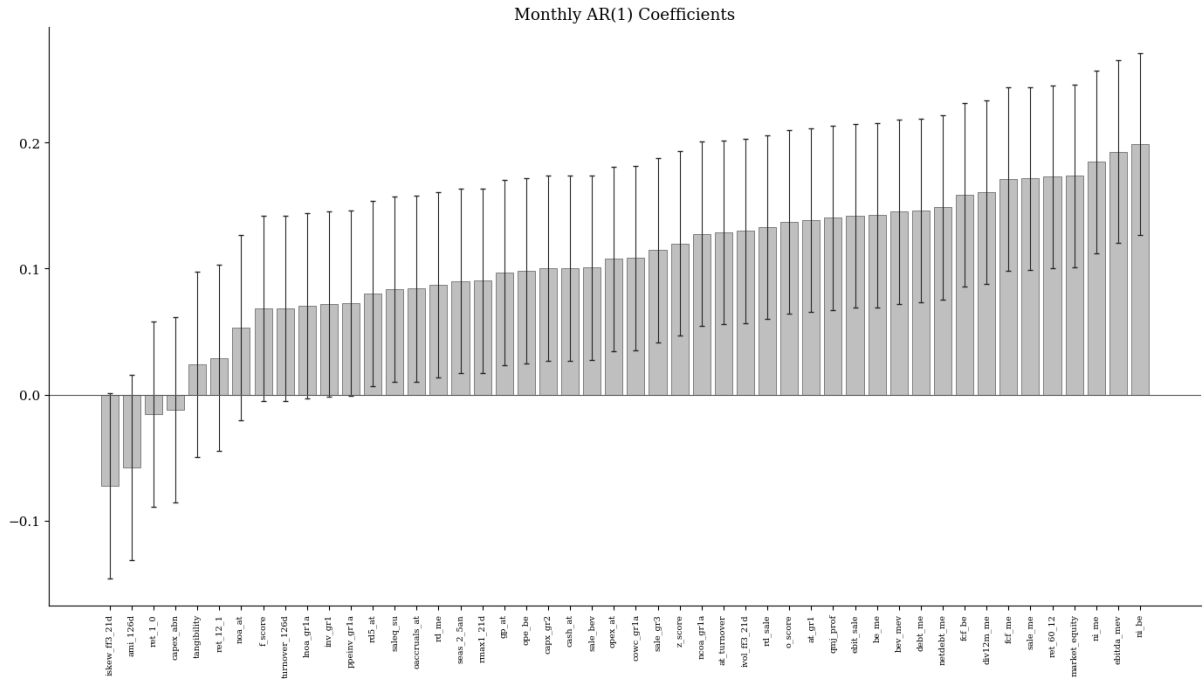


Figure 3

5.3 TSFM Performance & Own-Factor Alpha

The effectiveness of timing a factor can be assessed by regressing the scaled factor return on the raw factor. The scaling term s is given by equation (1) and the scaled return is given by equation (2), as we have discussed in the research methodology section. Table 2 provides an overview of the timed factor returns, reporting the annualized average excess return, Sharpe ratio, and own-factor alpha for each factor. The average excess return and Sharpe ratio are computed directly from the scaled factor returns, while the own-factor alpha measures the annualized excess return from timing the factor relative to a passive approach of just holding it. We have also included the aggregated TSFM factor. Since there is no “untimed” TSFM factor return, the dynamic TSFM return is regressed on an equal-weighted portfolio of raw factor returns. As with the individual factors, this isolates the contribution of the timing element.

Table 2: Timed Factor Returns: Average Excess Return, Sharpe Ratio, and Own-Factor Alpha

Factor	$\bar{R}_{ann}$	Sharpe _{ann}	α_{ann}
fcf_be	0.0860	0.7414	0.0593
TSFM_total	0.0834	0.6750	0.0705
fcf_me	0.0944	0.5889	0.0616
noa_at	0.0453	0.5876	0.0221

Continued on next page

Table 2 – continued

Factor	$\bar{R}_{ann}$	Sharpe _{ann}	α_{ann}
qmj_prof	0.0596	0.5859	0.0404
ni_be	0.0786	0.5773	0.0753
sale_bev	0.0407	0.5515	0.0276
ebitda_mev	0.0778	0.5302	0.0678
o_score	0.0498	0.5238	0.0441
ebit_sale	0.0627	0.5109	0.0590
sale_me	0.0815	0.5021	0.0763
ncoa_gr1a	0.0390	0.4938	0.0248
ni_me	0.0783	0.4931	0.0745
at_turnover	0.0377	0.4805	0.0268
netdebt_me	0.0707	0.4800	0.0702
debt_me	0.0726	0.4679	0.0727
rd_sale	0.0765	0.4545	0.0739
tangibility	0.0329	0.4448	0.0271
ope_be	0.0579	0.4426	0.0497
opex_at	0.0320	0.4365	0.0231
cash_at	0.0522	0.4253	0.0514
capx_gr2	0.0302	0.4219	0.0208
lnoa_gr1a	0.0355	0.4206	0.0228
div12m_me	0.0662	0.4121	0.0656
gp_at	0.0359	0.4089	0.0219
z_score	0.0368	0.3845	0.0378
ret_60_12	0.0391	0.3828	0.0386
bev_mev	0.0573	0.3799	0.0573
ivol_ff3_21d	0.0692	0.3716	0.0611
rd5_at	0.0535	0.3671	0.0455
be_me	0.0607	0.3642	0.0591
market_equity	0.0250	0.3603	0.0239
rd_me	0.0455	0.3441	0.0362
capex_abn	0.0167	0.3325	0.0095
oaccruals_at	0.0238	0.3124	0.0058
cowc_gr1a	0.0167	0.3107	0.0027
rmax1_21d	0.0537	0.2952	0.0521
ppeinv_gr1a	0.0238	0.2533	0.0175
f_score	0.0228	0.2387	0.0223
saleq_su	0.0160	0.2247	0.0155
at_gr1	0.0225	0.2228	0.0168
sale_gr3	0.0186	0.2169	0.0186
seas_2_5an	0.0136	0.1925	0.0095

Continued on next page

Table 2 – continued

Factor	$\bar{R}_{ann}$	Sharpe _{ann}	α_{ann}
turnover_126d	0.0311	0.1853	0.0313
ret_12_1	0.0344	0.1710	0.0403
inv_gr1	0.0113	0.1554	-0.0021
ami_126d	0.0020	0.0264	0.0046
ret_1_0	-0.0094	-0.0629	0.0011
iskew_ff3_21d	-0.0087	-0.1916	-0.0090

While [Gupta and Kelly \(2019\)](#) find that the aggregated TSFM strategy yields the best own-factor alpha due to the diversification benefits across factor signals, our results are slightly different. Five factors have a higher own-factor alpha than the aggregate TSFM strategy. This may indicate that during our sample period, the timing signal was particularly effective for specific factors, whereas the weaker-performing factors in the TSFM basket may have diluted the overall TSFM alpha. We do, however, observe that the aggregate TSFM portfolio achieves the second highest Sharpe ratio across all timed factors, only surpassed by `fcf_be`. This highlights the diversification benefits of aggregating across factor signals, even when some individual factors dominate the TSFM aggregate in terms of alpha.

A reasonable question from an investor’s point of view is why one should not simply hold the `fcf_be` factor passively rather than employing a dynamic timing strategy such as TSFM. After all, `fcf_be` achieves the highest Sharpe ratio of any single factor at 1.02, exceeding both the timed `fcf_be` Sharpe of 0.74 and the aggregate TSFM Sharpe of 0.68. Additionally, it has a higher average return of 8.95% compared to 8.34% for TSFM. This appears to make a compelling case for a passive single-factor strategy. However, several important considerations argue against this conclusion. First, the FF5 alpha of `fcf_be` is 4.97%, lower than the 7.49% achieved by the aggregate TSFM strategy as we will get back to later. This indicates that a larger part of the `fcf_be` return can be attributed to standard risk factor exposure, while TSFM generates a larger share of its return from pure timing skill that is not captured by the Fama-French factors. Second, and arguably more importantly, the outstanding performance of `fcf_be` is likely sample-specific. Our sample covers a period where value and profitability factors performed particularly well, and there is no guarantee that `fcf_be` would replicate such a strong performance over a different or longer sample period. In contrast, the TSFM strategy is diversified across 48 factors, making its performance less dependent on any single factor continuing to outperform. Finally, holding a single-factor portfolio exposes the investor to prolonged drawdown periods specific to that factor, whereas the aggregate TSFM strategy benefits from diversification across factors.

Regardless, the presence of significant alphas across the majority of factors—including TSFM—reinforces

the claim that factor momentum provides a distinct source of return that cannot be captured by static factor exposure alone.

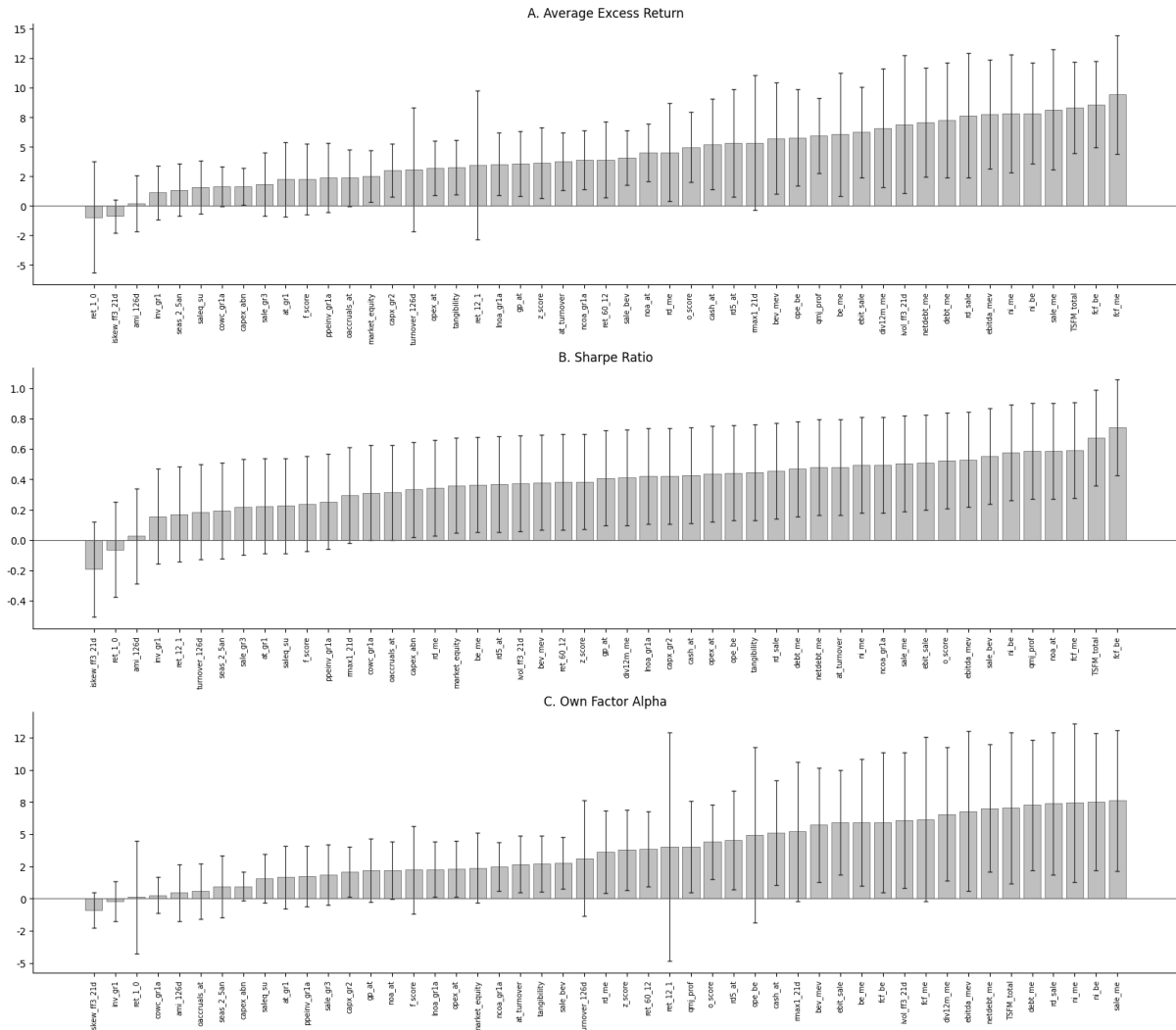


Figure 4: Average Excess Returns, Sharpe ratios, and own-factor alphas for timed factors. Panel A shows annualized average excess returns, Panel B shows annualized Sharpe ratios, and Panel C shows annualized own-factor alphas. Error bars represent 95% confidence intervals.

5.4 Boosting

We evaluate six boosting model specifications across all 48 factors over the out-of-sample period 1985–2024. Although our data begins in 1965, we require a minimum of 20 years (240 monthly observations) of training data before generating any out-of-sample predictions, which is why the evaluation period begins in 1985. The three boosting algorithms considered are XGBoost, CatBoost, and LightGBM. Each algorithm is implemented under two training window schemes. The first is an expanding window, where the model is trained on all available data up to the retraining date, with the training set growing over time. The second is a rolling fixed window, where the training set maintains a constant size of 240 months and moves forward 5 years at a time, ensuring that only the most recent data informs each prediction. We consider both schemes because they make different assumptions about how the factor dynamics evolve over time. The expanding window exploits the full data history available at each point in time, which could improve estimation precision but carries the risk of including stale relationships that no longer hold. On the other hand, the fixed window forces the model to only learn recent factor dynamics at the cost of discarding potentially informative older data. Comparing the two therefore enables us to assess whether factor return predictability is better captured by using long-run historical patterns or by focusing on recent market conditions. This comparison is especially important in the context of regression trees, where predictions tend to perform poorly when the test data falls outside the domain of the training data.

In both schemes, hyperparameters are re-tuned periodically via cross-validation, with the tuned parameters including the number of estimators, maximum tree depth, minimum child weight, column subsampling ratio, and L2 regularization strength. Additionally, we create ensemble models as averages of the six base models. We have three of these: one which is an ensemble of only the three fixed rolling window models, one which is an ensemble of only the three expanding window models, and finally an ensemble model of all six base models. This results in nine models in total, allowing us to compare both the choice of boosting algorithm and the effect of training window design on out-of-sample predictive performance.

5.4.1 Predictive Performance

A common method for evaluating predictive performance is out-of-sample (OOS) root mean squared error (RMSE), defined as the square root of the average squared prediction error on data outside the training set. A lower RMSE value implies a better prediction. Table 3 reports the mean, minimum and maximum percentage decrease in RMSE across factors compared to the baseline model that simply uses last month's return as the prediction for next month's return. The mean OOS R^2 for different models is

also reported.

Table 3: Predictive Performance of Boosting Models

Model	% Decrease in RMSE			Mean R^2
	Mean	Min	Max	
XGBoost (Expanding)	25.08%	20.37%	32.84%	-0.0043
XGBoost (Fixed)	24.76%	18.76%	32.15%	-0.0127
LightGBM (Expanding)	25.18%	21.36%	32.14%	-0.0017
LightGBM (Fixed)	25.01%	20.16%	32.44%	-0.0060
CatBoost (Expanding)	25.24%	20.87%	32.61%	0.0000
CatBoost (Fixed)	25.12%	20.61%	32.40%	-0.0031
Ensemble (Expanding only)	25.32%	21.28%	32.70%	0.0021
Ensemble (Fixed only)	25.19%	20.35%	32.51%	-0.0012
Ensemble	25.36%	20.98%	32.69%	0.0031
Lagged Return Baseline	0.00%	0.00%	0.00%	-0.7955

The result suggests that machine-learning adds value in predicting returns. The minimum decrease in RMSE is 18.76% for any single factor, while the maximum is 32.84%. All models perform relatively similarly, but we can see that expanding window models accomplish a slightly higher decrease in RMSE than fixed window for all model specifications. This indicates that more training data is beneficial for prediction, and that the additional observations from earlier periods are useful rather than noise. An alternative way to measure model performance is to look at OOS R^2 , which is defined in equation 11. We only compute this for data outside the training set.

$$R^2 = 1 - \frac{\sum_{t=1}^T (y_t - \hat{y}_t)^2}{\sum_{t=1}^T (y_t - \bar{y})^2}, \quad (11)$$

The mean OOS R^2 values are around zero for all boosting models, ranging from -0.0127 for XGBoost (Fixed) to 0.0031 for the full Ensemble. These values are relatively small but, as [Kelly et al. \(2024\)](#) states, that does not necessarily mean that the prediction is bad. Moreover, the lagged return baseline has a R^2 of -0.7955 , so even R^2 values close to zero represent a substantial improvement. It must also be noted that this is a mean OOS R^2 across all factors, with OOS R^2 for the expanding ensemble ranging from -2.78% to 4.28% for individual factors as an example. The ensemble models perform at least as well as any individual algorithm, consistent with the notion that combining predictions across models has the benefit of reducing variance.

5.4.2 Optimal Hyperparameters

After performing cross-validation for each factor every 5 years we end up with a list of optimal hyperparameters, optimal in this case being the hyperparameters that minimize the cross-validation mean squared error (MSE). Training of the models was also done with a mean squared error loss. For our optimization to find optimal hyperparameters we first had to define the boundaries in which we searched, these are defined in Table 4.

Table 4: Hyperparameter search spaces for the boosting models

Model	Hyperparameter	Search space	Sampling scale
XGBoost	max_depth	$\{2, \dots, 5\}$	Integer
	reg_lambda	$[10^{-5}, 1]$	Log-uniform
	n_estimators	$\{50, \dots, 250\}$	Integer
	min_child_weight	$\{3, \dots, 10\}$	Integer
	colsample_bytree	$[0.5, 1.0]$	Uniform
LightGBM	num_leaves	$\{10, \dots, 50\}$	Integer
	reg_lambda	$[10^{-5}, 1]$	Log-uniform
	n_estimators	$\{50, \dots, 250\}$	Integer
	min_child_weight	$\{3, \dots, 10\}$	Integer
	colsample_bytree	$[0.5, 1.0]$	Uniform
CatBoost	depth	$\{2, \dots, 5\}$	Integer
	l2_leaf_reg	$[10^{-5}, 1]$	Log-uniform
	n_estimators	$\{50, \dots, 250\}$	Integer
	min_data_in_leaf	$\{3, \dots, 10\}$	Integer
	rsm	$[0.5, 1.0]$	Uniform

This is by no means the definitive best search space. The search space was chosen based on a combination of previous experience, algorithm documentation recommendations, computational resource constraints, and what we wanted to achieve. An example of selecting the search space based on what we wanted to achieve is forcing some complexity and potential interactions between features through not allowing a max_depth of 1 and setting the lower bound of boosting rounds to 50. The logic behind this is that if these hyperparameters are too flexible then other hyperparameters can help in controlling the overfit, but we still maintain some degree of complexity.

Taking the average of the optimal results across all factors and time periods for each gradient boosting

implementation resulted in Table 5. Note that `l2_leaf_reg` and `rsm` for CatBoost have been combined with `reg_lambda` and `colsample_bytree`, respectively, as they are equivalent.

Table 5: Mean selected hyperparameters across all prediction tasks

Model	colsample_bytree	max_depth	min_child_weight	n_estimators	reg_lambda	num_leaves	min_data_in_leaf
XGBoost (Fixed)	0.663	2.836	7.346	74.643	0.131	–	–
XGBoost (Expanding)	0.652	2.646	7.094	73.466	0.147	–	–
LightGBM (Fixed)	0.634	–	6.466	73.258	0.158	31.602	–
LightGBM (Expanding)	0.602	–	6.677	62.050	0.172	30.417	–
CatBoost (Fixed)	0.749	3.547	–	109.883	0.155	–	6.740
CatBoost (Expanding)	0.735	3.586	–	125.138	0.121	–	6.677

This gives us some insight into differences between algorithms and also insight into whether the bounds we set were appropriate. It must be taken into account that this is not a perfect comparison due to the different algorithms taking different approaches to growing regression trees, but it is still interesting nonetheless. This table does also not show the whole picture, as it only shows the first moment of the distributions of hyperparameters, we still regard this as acceptable as it is not the focus of this thesis.

The first interesting thing to notice is that the hyperparameters for XGBoost and LightGBM are quite similar, whilst CatBoost allows more flexibility in its hyperparameters. This could be due to CatBoost growing symmetric trees, which can be thought of as less prone to overfitting as they impose more constraints on what variables can be split on. Secondly, `n_estimators` for XGBoost and LightGBM are quite close to the lower bound, this may indicate that the lower bound is too high, looking a bit deeper into the data we see that this lower bound is hit many times, but for stated reasons we accept this. The same logic applies to `max_depth` for XGBoost.

Taking a step back we note that endless time can be spent on tuning and playing around with hyperparameters, we could even add many more hyperparameters to tune as the chosen ones are only a small subset of what we had available. For the purpose of this thesis we are satisfied with the compromises made and the resulting output.

5.5 MLTSFM Performance

5.5.1 Portfolio Performance

Table 6 reports the annualized average excess return, Sharpe ratio, and own-factor alpha for each of the nine strategies over the out-of-sample period.

All strategies deliver positive and significant returns (see Figure 5), with annualized average excess returns ranging from 6.46% for XGBoost (Fixed) to 8.64% for the full ensemble. The Sharpe ratios range

from 0.69 to 0.89, all of which are statistically significant. The expanding window models consistently outperform their fixed window counterparts within each algorithm, in line with the result from the predictive performance analysis that additional historical data improves out-of-sample performance. XGBoost (Fixed) is the weakest individual model with a Sharpe ratio of 0.69, while XGBoost (Expanding) achieves 0.89, the highest among the six base models. The ensemble strategies perform well, with the expanding-only ensemble achieving the highest overall Sharpe ratio of 0.86 out of the ensembles, and the full ensemble delivering the highest average excess return of 8.64%.

Table 6: Average excess returns, Sharpe ratios and Own-Factor Alphas for MLTSFM strategies.

Strategy	$\bar{R}_{ann}$	Sharpe _{ann}	α_{ann}
XGBoost (Fixed)	0.0646	0.6890	0.0469
XGBoost (Expanding)	0.0736	0.8923	0.0707
LightGBM (Fixed)	0.0834	0.8330	0.0587
LightGBM (Expanding)	0.0755	0.8480	0.0601
CatBoost (Fixed)	0.0731	0.7705	0.0626
CatBoost (Expanding)	0.0779	0.8560	0.0710
Ensemble (Fixed only)	0.0787	0.7921	0.0600
Ensemble (Expanding only)	0.0785	0.8642	0.0704
Ensemble (Full)	0.0864	0.8512	0.0696

In Table 6, we report only the alphas of the nine aggregate MLTSFM strategies relative to the equal-weighted raw factor benchmark. There are two main reasons for this. First, the primary focus of our analysis is the performance of aggregate factor momentum portfolios rather than individual timed factors. Second, individual MLTSFM factor returns are not directly comparable to their TSFM counterparts, since the ML-based timing signals generally imply considerably smaller weights than the traditional TSFM signals because the forecasts are substantially less volatile than realized one-month returns, while both signals are scaled by realized factor volatility. As a result, the individual MLTSFM factor portfolios are effectively deleveraged relative to TSFM, making a direct comparison of individual-factor alphas difficult to interpret. For reference, the three largest individual MLTSFM own-factor alphas are obtained by `market_equity`, `netdebt_me`, and `fcf_be`, with annualized alphas of 1.41%, 1.26%, and 0.99%, respectively. These are substantially lower than the alphas of the aggregate MLTSFM strategies. Although one could rescale the individual portfolios to make their exposures more comparable, we do not pursue this further because the thesis focuses on TSFM/CSFM-style aggregate portfolios rather than individual factor timing strategies.

All nine MLTSFM aggregate portfolios deliver own-factor alphas between 4.69% and 7.10%, confirming

that the timing signal delivers excess returns beyond what can be attributed to passive factor exposure. Expanding window models consistently outperform their fixed window counterparts across all three algorithms, with CatBoost (Expanding) achieving the highest alpha of 7.10%, closely followed by XGBoost (Expanding) at 7.07% and the expanding-only ensemble at 7.04%. This pattern reinforces the finding from the predictive performance analysis that access to a longer training sample improves the quality of the timing signal. The full ensemble achieves an alpha of 6.96%, slightly below the top expanding window models but above all fixed window specifications, suggesting that diversification across algorithms partially compensates for the inclusion of the weaker fixed window models. XGBoost (Fixed) produces the lowest alpha at 4.69%, consistent with its weaker performance in the portfolio performance analysis.

5.5.2 Comparison with TSFM

Figure 5 compares average returns, Sharpe ratios and alphas on both the equal-weighted factor portfolio and FF5 for all MLTSFM strategies and TSFM.

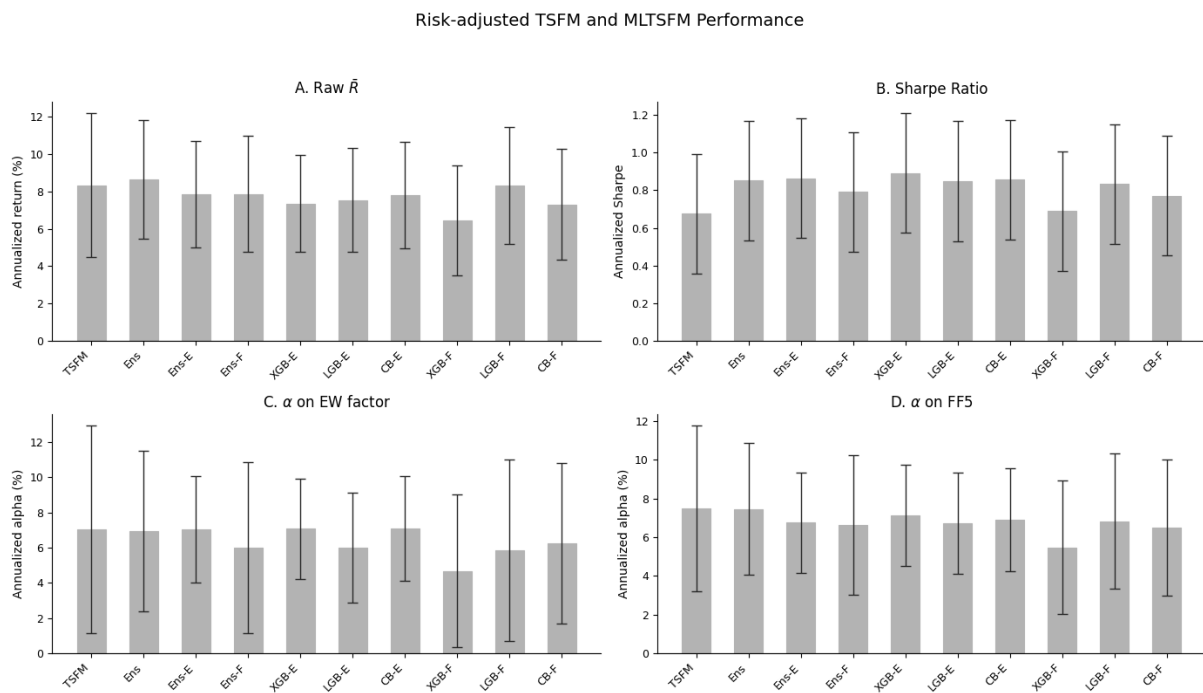


Figure 5

There are several interesting observations. Firstly, the MLTSFM strategies generally match or exceed the TSFM baseline across all four panels, suggesting that replacing the simple TSFM strategy with an ML-predicted signal does not reduce performance and in most cases improves it. In terms of raw average return (Panel A), the full ensemble delivers the highest annualized return at 8.64%, compared to 8.34% for TSFM. The Sharpe ratio comparison in Panel B is particularly interesting. The TSFM baseline achieves a Sharpe of 0.68, while *all* MLTSFM strategies exceed this, with several expanding window

models and ensemble strategies reaching Sharpe ratios above 0.85. When looking at Panels C and D, we observe that the baseline TSFM performs well, also in comparison to the MLTSFM portfolios. It actually has the highest FF5 alpha at 7.49%, narrowly beating the full ensemble model (7.46%). It is, however, important to note that the confidence interval is considerably larger for TSFM than the full ensemble. This is a recurring theme we see across panels C and D; most of the strategies have substantially smaller confidence intervals than TSFM. Smaller confidence intervals indicate that the MLTSFM strategies exhibit more stable and consistent returns over time, meaning the estimated alphas are more precisely measured and less sensitive to specific periods in the sample. In contrast, the wider confidence intervals for TSFM demonstrate greater variability in its timing performance. While the estimate is competitive, the signal is noisier and the outperformance is thus statistically less reliable. This distinction is important in a practical setting: an investor evaluating these strategies not only cares about the expected alpha, but also about the certainty with which that alpha can be attributed to skill rather than chance. From this perspective, the MLTSFM strategies offer an improvement over the simple momentum-based timing rule.

5.6 MLCSFM Performance

5.6.1 Portfolio Performance

Up to this point, our analysis has been limited to TSFM and the extension MLTSFM. We will now shift our attention to Cross-Section Factor Momentum (CSFM) and Machine-Learned Cross-Section Factor Momentum (MLCSFM). This is an alternative way of forming factor momentum portfolios introduced by [Gupta and Kelly \(2019\)](#). We form these portfolios by going long an equally-weighted portfolio of factors with above median recent returns (predicted returns for MLCSFM) and short an equally-weighted portfolio of factors with below median recent returns (predicted returns for MLCSFM). Our framework is the same as with MLTSFM for MLCSFM; we have six base models and three ensemble models, in total nine different implementations. Table 7 presents their initial performance results.

Table 7: Average excess returns, Sharpe ratios and Own-Factor Alphas for MLCSFM strategies.

Strategy	$\bar{R}_{ann}$	Sharpe _{ann}	α_{ann}
MLCSFM XGBoost (Fixed)	0.0344	0.6149	0.0160
MLCSFM XGBoost (Expanding)	0.0434	0.9106	0.0282
MLCSFM LightGBM (Fixed)	0.0430	0.7189	0.0216
MLCSFM LightGBM (Expanding)	0.0434	0.7589	0.0209
MLCSFM CatBoost (Fixed)	0.0362	0.6773	0.0186
MLCSFM CatBoost (Expanding)	0.0483	0.8886	0.0344

Continued on next page

Table 7 – continued

Strategy	$\bar{R}_{ann}$	Sharpe _{ann}	α_{ann}
MLCSFM Ensemble (Fixed only)	0.0408	0.6858	0.0188
MLCSFM Ensemble (Expanding only)	0.0508	0.8915	0.0320
MLCSFM Ensemble	0.0481	0.8338	0.0273

All strategies deliver positive annualized average returns, ranging from 3.44% for XGBoost (Fixed) to 5.08% for the expanding-only ensemble. The Sharpe ratios range from 0.61 to 0.91 across the MLCSFM strategies, most notably XGBoost (Expanding), which achieves the highest Sharpe ratio of 0.91. The own-factor alphas are all positive, though the lower bound of the confidence interval lies below zero for five out of the nine strategies: Ensemble (fixed), LightGBM (expanding), XGBoost (fixed), LightGBM (fixed) and CatBoost (fixed) (see figure 6). As we saw with MLTSFM, the expanding window models seem to perform slightly better than their fixed window counterparts. Every expanding window model performs better than its fixed window equivalent on both average return and Sharpe ratio. The alpha is also higher for the expanding models for all but one implementation (LightGBM). This reinforces the claim that a longer training history improves signal quality.

Comparing MLCSFM directly with the corresponding MLTSFM strategies, the MLTSFM portfolios consistently deliver higher average returns across all model specifications, but this is of course uninformative without considering risk. When it comes to Sharpe ratio the results are more varied. For XGBoost (Expanding), CatBoost (Expanding), and the expanding-only ensemble, the MLCSFM version obtains a marginally higher Sharpe ratio than its MLTSFM counterpart, though the differences are small. For the remaining specifications, MLTSFM achieves higher values on both metrics. The two frameworks therefore appear broadly comparable in terms of risk-adjusted performance, with neither consistently outperforming the other across all metrics and model specifications. What can be said is that MLTSFM tends to generate higher average returns, while MLCSFM occasionally produces better Sharpe ratios, particularly for expanding window models.

5.6.2 Comparison with CSFM

Figure 6 presents the four risk-adjusted performance metrics across all MLCSFM strategies alongside the CSFM baseline. The CSFM baseline achieves a considerably higher average return than all MLCSFM strategies, while its Sharpe ratio of 0.62 falls below most MLCSFM implementations. Panels C and D show that the baseline also achieves higher point estimates on both alpha measures. However, as with the MLTSFM results, the confidence intervals for the CSFM baseline are substantially wider than those of the MLCSFM strategies. This is especially prominent in panel C, where the lower bound of the confidence

Risk-adjusted CSFM and MLCSFM Performance

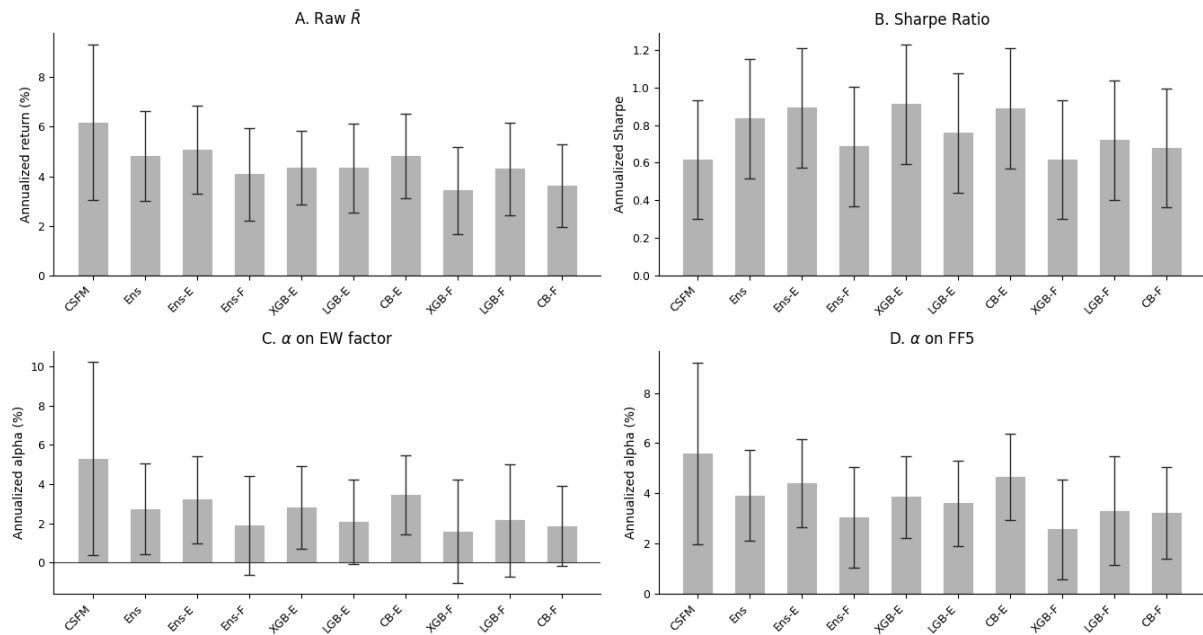


Figure 6

interval approaches zero despite the alpha being 5.29%, indicating uncertainty around the estimate. The same insight from the MLTSFM results applies here: the narrower confidence intervals of the MLCSFM strategies reflect more stable and consistent performance over time, making the estimated alphas more statistically reliable even if they are somewhat lower in magnitude. The results in panel D demonstrate that the returns of all specifications are not explained by standard risk factors, supporting the interpretation that cross-sectional factor momentum represents a persistent source of excess return.

5.6.3 Comparison of Momentum Strategies

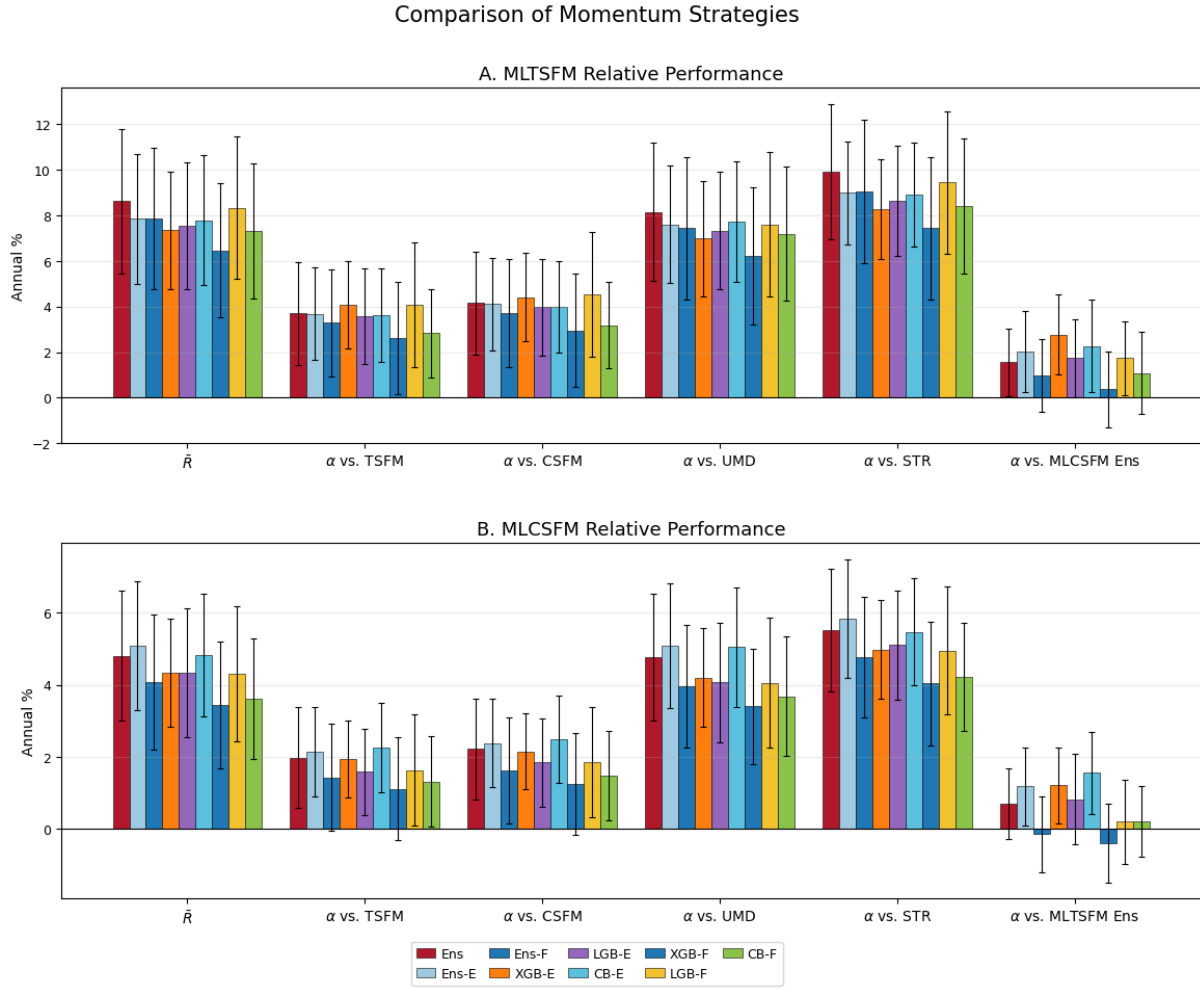


Figure 7

Figure 7 provides a comprehensive comparison of MLTSFM and MLCSFM strategies across multiple alpha benchmarks. Beginning with alpha relative to the TSFM baseline (second group of bars), MLTSFM strategies consistently generate higher alphas. Most values are clustered slightly below 4% and all nine are statistically significant. The MLCSFM strategies also generate positive alphas against TSFM, but at lower magnitudes of around 2%. Additionally, two out of nine are not statistically significantly different from zero. We observe a similar pattern when benchmarking against the CSFM baseline. Every MLTSFM alpha is significant and higher in magnitude, while one MLCSFM strategy produces an insignificant alpha. This indicates that both sets of strategies contain timing information beyond their respective baselines, but the MLTSFM signal is more reliably distinguishable from its benchmark.

When benchmarking against the UMD factor, both MLTSFM and MLCSFM strategies produce significant alphas across all nine specifications, confirming that the returns generated by both frameworks are not simply capturing stock-level momentum and represent a distinct source of return. The alpha versus

the short-term reversal factor (STR) yields the highest alpha magnitudes of any benchmark in the figure for both frameworks. The final group of bars is perhaps the most informative for directly comparing the two frameworks. When MLTSFM strategies are benchmarked against the MLCSFM ensemble, all nine alphas are positive though not all of them are statistically significant. The implication is that the MLTSFM approach contains additional information beyond what is captured by the MLCSFM ensemble. When performing the reverse comparison — benchmarking MLCSFM strategies against the MLTSFM ensemble — two strategies generate negative alphas and the majority are statistically insignificant. This asymmetry suggests that the MLTSFM ensemble to some extent encompasses the cross-sectional signal while the MLCSFM ensemble does not fully capture the time series signal.



Figure 8

Looking at some of the reverse relationships in figure 8 we can see that the machine learned portfolios are largely able to explain the excess return of TSFM and CSFM as the majority of their alphas are statistically insignificant. This relationship will play a role when we later combine them into ex-post optimal portfolios.

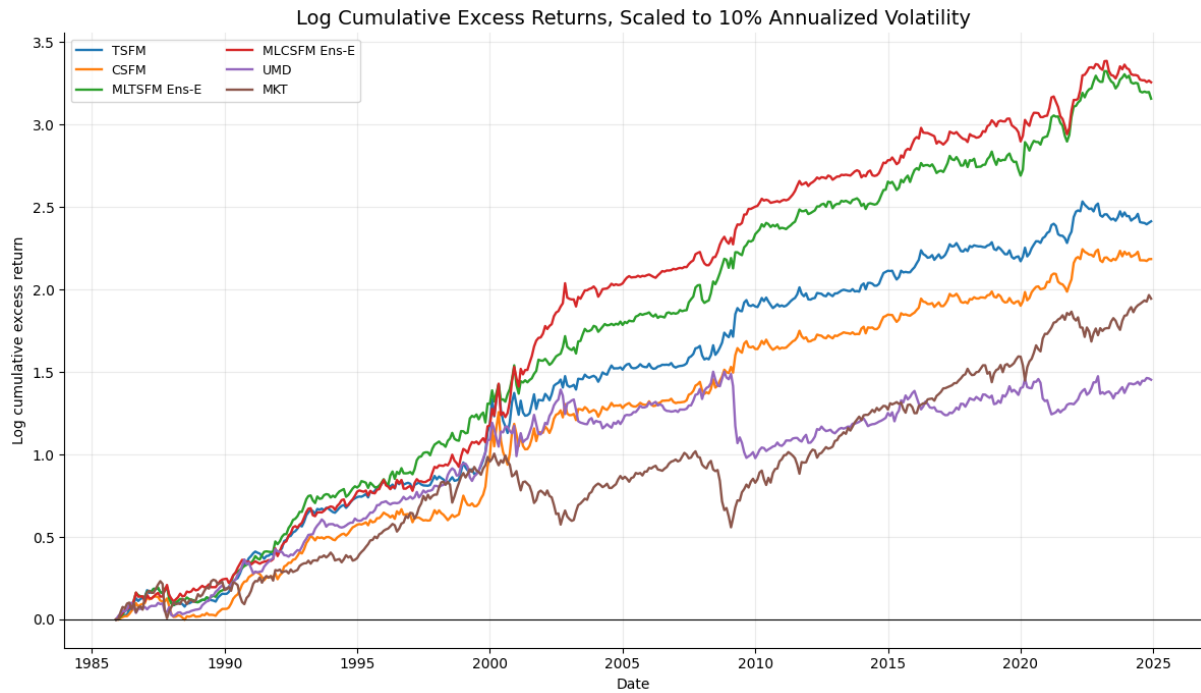


Figure 9

Figure 9 plots the log cumulative returns of all of our main factor momentum strategies together with UMD and the market return as a benchmark. To avoid cluttering the figure, we only plot the cumulative return of the expanding ensemble, which serves as a representative specification of the expanding-window MLTSFM results. All return series have been rescaled to have an ex-post volatility of 10% for a fair comparison. Here we can clearly see that within each class of prediction used (machine-learned vs normal) the behavior between the cross-section and time-series portfolio is quite similar, which is also backed up by their correlations (Table 10). It is also notable that the factor momentum strategies do not experience the large drawdowns of the market and UMD, and share a consistently steep slope. The higher Sharpe ratios of the ML portfolios are also highlighted here by their consistently steeper slope.

5.7 Pooled MLTSFM/TSFM

5.7.1 Performance measures

The pooled version of TSFM/MLTSFM presents us with the interesting opportunity to look at what happens when we have a net factor bias. Figure 10 presents the pooled versions of TSFM/MLTSFM. We have also decided to include the equal weight factor portfolio due to similarities between it and the pooled MLTSFM.

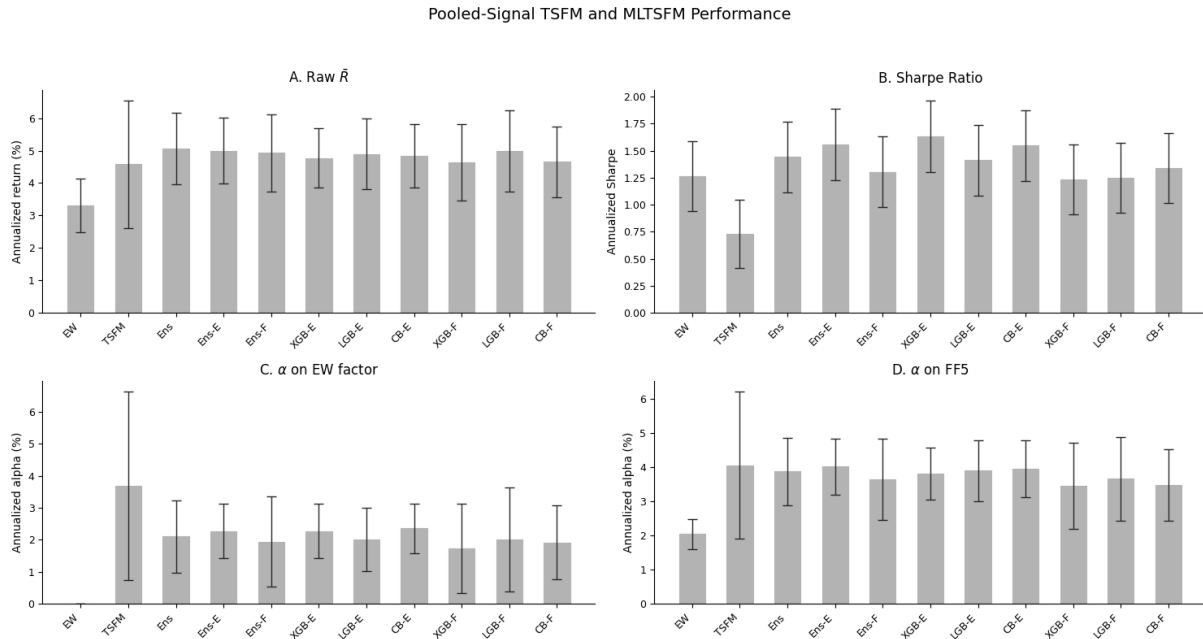


Figure 10

Comparing the normal TSFM portfolio with the pooled version we see a marginal improvement in Sharpe ratio, going from 0.68 to 0.73. We also see positive and statistically significant own-factor alpha and FF5 alpha as before. The most interesting development is for the MLTSFM portfolios where we see a large increase in Sharpe ratio across the board, with the expanding ensemble Sharpe going from 0.86 to 1.56, a near doubling. The point estimates of alpha relative to the equal-weighted factor portfolio are also notable, as the pooled MLTSFM portfolios have a consistently lower alpha than the pooled TSFM portfolio, which points in the direction of the equal-weight portfolio explaining more of the excess return in the pooled MLTSFM portfolios. Although it must be noted that the confidence interval for pooled TSFM is quite large, and all of the pooled MLTSFM equal-weighted alphas are positive and statistically significant at the 5% level. The FF5 factor model is also largely unable to explain the performance of the pooled portfolios, as the alphas on it are positive and highly statistically significant. It must also be noted that the confidence interval for pooled TSFM is the largest.

5.7.2 Factor weights

Examining the factor weights in more detail, we can start to understand why we see these results. Table 8 presents factor-level exposures, and reveals an important pattern. Note that net factor bias is not only determined by the amount of positive/negative signals, but also the size of the positive/negative signals, as large signals receive greater weight in pooled portfolios.

Table 8: Pooled factor-level exposure summary

Strategy	Long mean	Short mean	Net mean	Net min	Net max
EW Factor	1.0000	0.0000	1.0000	1.0000	1.0000
Pooled TSFM	0.5758	-0.4242	0.1517	-0.6290	0.8607
Pooled MLTSFM Ensemble	0.9362	-0.0638	0.8724	0.4995	1.0000
Pooled MLTSFM Ensemble (Expanding only)	0.9407	-0.0593	0.8813	0.3728	1.0000
Pooled MLTSFM Ensemble (Fixed only)	0.9128	-0.0872	0.8255	0.3759	1.0000
Pooled MLTSFM XGBoost (Expanding)	0.9368	-0.0632	0.8735	0.2586	1.0000
Pooled MLTSFM LightGBM (Expanding)	0.9130	-0.0870	0.8259	0.3926	0.9902
Pooled MLTSFM CatBoost (Expanding)	0.9394	-0.0606	0.8789	0.2502	1.0000
Pooled MLTSFM XGBoost (Fixed)	0.9031	-0.0969	0.8062	0.2023	1.0000
Pooled MLTSFM LightGBM (Fixed)	0.8751	-0.1249	0.7503	0.0711	0.9828
Pooled MLTSFM CatBoost (Fixed)	0.9240	-0.0760	0.8480	0.2863	1.0000

From this table we can gather that the pooled MLTSFM portfolios have a majority of the time a net positive factor bias, which makes sense since our predictions are positively biased. This pushes them closer to the EW factor portfolio as it is long all of the factors all of the time. It must be noted that it is not equal to the EW portfolio as the weights are not equal and pooled MLTSFM does allow for shorting factors. Pooled TSFM is more balanced in its approach to being long and short factors, although it does also have a positive net factor bias. As we see in figure 10 having an equal positive weight on all factors is a strategy that does well, with it having a Sharpe ratio of 1.26, so it makes sense that since pooled MLTSFM portfolios resemble it more closely they will have a higher Sharpe than pooled TSFM. Looking at table 9 we can clearly see that there is a difference between the pooled MLTSFM expanding ensemble³ and the EW portfolio.

Table 9: Factor weight summary for Pooled MLTSFM Ens-E

Factor	Mean w.	Mean abs. w.	Min w.	Max w.	Share pos.	Share neg.
cowc_gr1a	0.0457	0.0457	0.0083	0.1121	1.0000	0.0000
fcf_be	0.0383	0.0385	-0.0188	0.1033	0.9915	0.0085
noa_at	0.0367	0.0367	0.0107	0.0801	1.0000	0.0000
oaccruals_at	0.0364	0.0364	-0.0002	0.0818	0.9979	0.0021

Continued on next page

³The expanding ensemble is chosen as an example; the remaining pooled MLTSFM specifications are also different from the EW portfolio.

Table 9 – continued

Factor	Mean w.	Mean abs. w.	Min w.	Max w.	Share pos.	Share neg.
inv_gr1	0.0331	0.0331	0.0064	0.0826	1.0000	0.0000
fcf_me	0.0312	0.0314	-0.0186	0.1165	0.9851	0.0149
ppeinv_gr1a	0.0309	0.0309	0.0105	0.0667	1.0000	0.0000
lnoa_gr1a	0.0306	0.0306	0.0054	0.0577	1.0000	0.0000
capex_abn	0.0296	0.0296	0.0004	0.1036	1.0000	0.0000
ncoa_gr1a	0.0292	0.0292	-0.0023	0.0836	0.9979	0.0021
f_score	0.0287	0.0290	-0.0256	0.0630	0.9851	0.0149
ret_1_0	0.0269	0.0270	-0.0140	0.0888	0.9829	0.0171
qmj_prof	0.0252	0.0252	-0.0040	0.0534	0.9915	0.0085
capx_gr2	0.0243	0.0246	-0.0111	0.0647	0.9680	0.0320
ret_12_1	0.0238	0.0246	-0.0281	0.0566	0.9595	0.0405
sale_bev	0.0234	0.0236	-0.0251	0.0597	0.9851	0.0149
ebitda_mev	0.0232	0.0239	-0.0171	0.0873	0.9446	0.0554
seas_2_5an	0.0225	0.0242	-0.0281	0.0625	0.9168	0.0832
saleq_su	0.0220	0.0224	-0.0151	0.0841	0.9680	0.0320
ope_be	0.0220	0.0225	-0.0173	0.0606	0.9616	0.0384
ni_me	0.0183	0.0206	-0.0404	0.0596	0.9019	0.0981
rd_me	0.0179	0.0183	-0.0167	0.1311	0.9680	0.0320
rmax1_21d	0.0176	0.0183	-0.0295	0.0495	0.9510	0.0490
o_score	0.0169	0.0176	-0.0136	0.0508	0.9318	0.0682
opex_at	0.0162	0.0172	-0.0269	0.0666	0.9211	0.0789
gp_at	0.0143	0.0152	-0.0325	0.0456	0.9488	0.0512
ivol_ff3_21d	0.0139	0.0145	-0.0107	0.0453	0.9275	0.0725
at_turnover	0.0130	0.0140	-0.0269	0.0348	0.9339	0.0661
ni_be	0.0128	0.0168	-0.0434	0.0686	0.8230	0.1770
sale_me	0.0127	0.0184	-0.0489	0.0894	0.7932	0.2068
tangibility	0.0127	0.0143	-0.0234	0.0761	0.8806	0.1194
cash_at	0.0125	0.0142	-0.0511	0.0739	0.9296	0.0704
ret_60_12	0.0119	0.0129	-0.0164	0.1084	0.8422	0.1578

Continued on next page

Table 9 – continued

Factor	Mean w.	Mean abs. w.	Min w.	Max w.	Share pos.	Share neg.
at_gr1	0.0119	0.0164	-0.0359	0.0739	0.7633	0.2367
be_me	0.0114	0.0152	-0.0349	0.0562	0.7932	0.2068
iskew_ff3_21d	0.0106	0.0125	-0.0315	0.0418	0.8849	0.1151
ami_126d	0.0106	0.0245	-0.0764	0.0853	0.6802	0.3198
market_equity	0.0102	0.0260	-0.0785	0.0918	0.6162	0.3838
bev_mev	0.0081	0.0142	-0.0512	0.0544	0.7505	0.2495
netdebt_me	0.0079	0.0127	-0.0529	0.0771	0.6716	0.3284
div12m_me	0.0072	0.0123	-0.0397	0.0439	0.7527	0.2473
ebit_sale	0.0071	0.0118	-0.0321	0.0477	0.7484	0.2516
rd5_at	0.0066	0.0097	-0.0470	0.0359	0.7676	0.2324
z_score	0.0063	0.0107	-0.0329	0.0354	0.6972	0.3028
turnover_126d	0.0054	0.0065	-0.0165	0.0284	0.8380	0.1620
sale_gr3	0.0039	0.0089	-0.0342	0.0397	0.6844	0.3156
rd_sale	0.0031	0.0075	-0.0303	0.0501	0.6482	0.3518
debt_me	-0.0031	0.0097	-0.0544	0.0506	0.4627	0.5373

In this table the difference compared to the EW portfolio becomes apparent, with weights being able to deviate significantly from 1/48. The factors that receive the largest mean weight also tend to be the ones that perform the best on an individual risk-adjusted level, the reverse also holds for the lowest weight factors, although this is not a strict rule. Most factors also receive negative weights in at least some periods, while only six are assigned positive weights throughout the sample. We can still see that negative weights are rather rare, as debt_me is the only factor whose weight is more often negative than positive.

5.7.3 Pooled MLTSFM relative performance

Comparing pooled MLTSFM with standard MLTSFM, MLCSFM, TSFM, and benchmark past-return factor portfolios is central to evaluating whether the proposed methodology adds value. As done previously, we can gauge whether there is any incremental benefit from including pooled MLTSFM in combination with other portfolios such as normal MLTSFM. We gauge this by looking for excess return unexplained by risk factors in our regression, in other words positive and significant alpha. Figure 11 presents these

results.

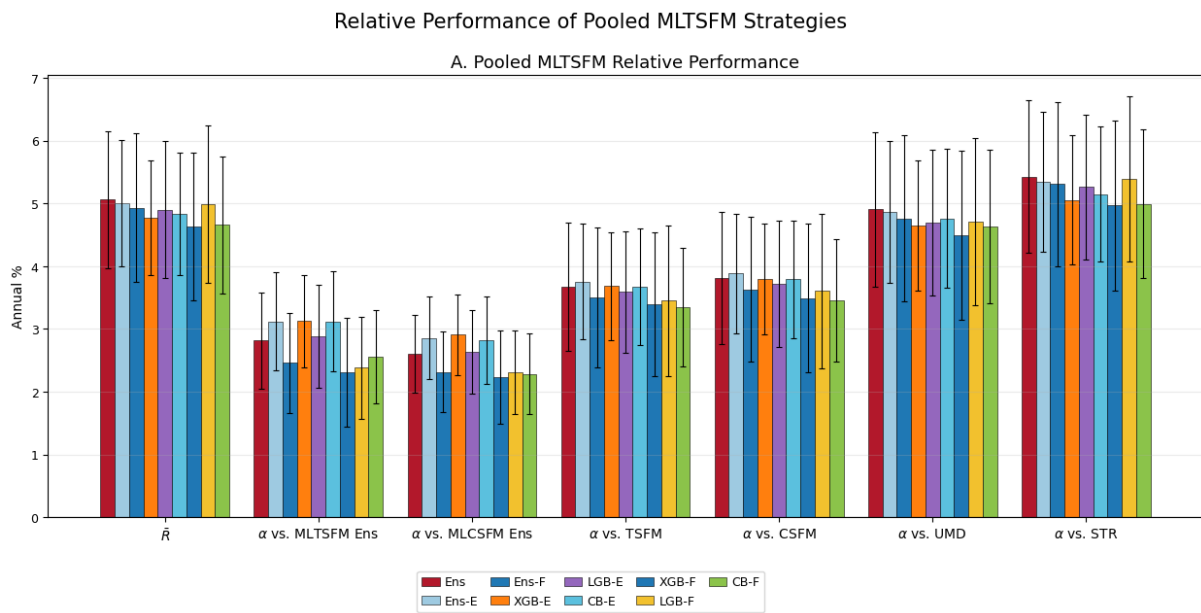


Figure 11

All of the alphas presented are positive and highly significant, indicating that these portfolios are not able to explain all of the excess return of pooled MLTSFM. The alphas on the normal machine learned portfolios are smaller than for TSFM and CSFM, which is natural since the same predictions have been used to create the pooled and normal variants, but they are still highly statistically significant. UMD does not manage to explain any of the performance of pooled MLTSFM as their alphas are essentially equal to the excess return of pooled MLTSFM. The STR alphas actually increase slightly above the excess return, potentially explained by their negative correlation. The same pattern for alphas is seen by [Gupta and Kelly \(2019\)](#) when they regress one-month TSFM on UMD and STR.

Next we do the reverse regressions⁴ for the same portfolios in figure 12.

⁴Not fully reverse as we are only regressing on the pooled MLTSFM total ensemble portfolio.

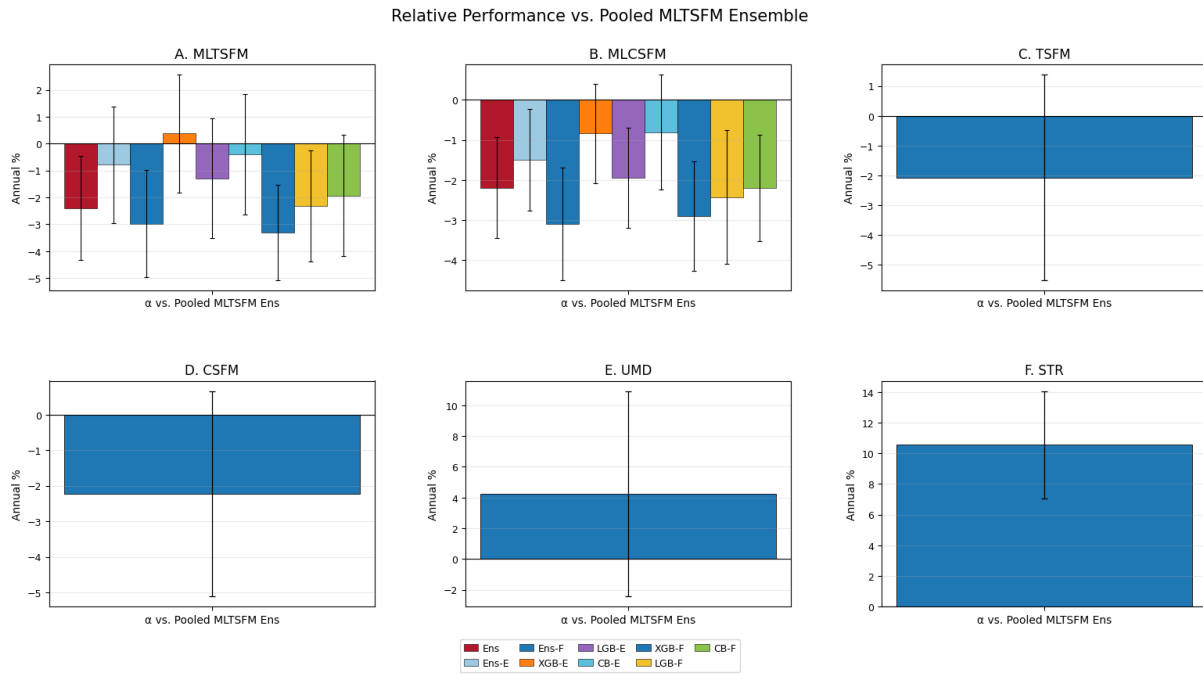


Figure 12

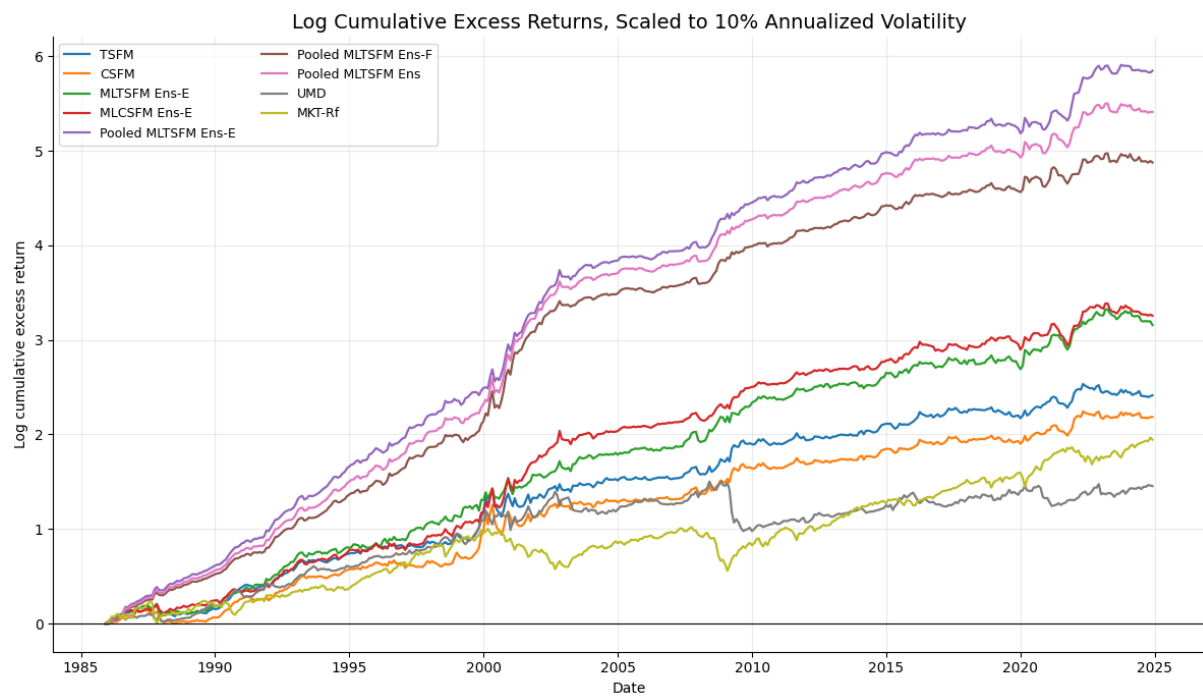


Figure 13

In figure 12 we see that pooled MLTSFM is more efficient at exploiting factor momentum compared to normal MLTSFM and MLCSFM as nearly all of the alphas are negative, and the majority are statistically significant. The exception is the normal MLTSFM XGB-E portfolio, which we have consistently seen as one of the best performing normal MLTSFM portfolios. The alphas are still negative for TSFM and CSFM,

although not significant. The alpha for UMD is positive, and for STR it is positive and significant, so we should expect these to have a positive weight when combining them with pooled MLTSFM in a mean-variance optimal portfolio framework.

Adding pooled MLTSFM to the cumulative log return plot in figure 13 we visually see the large difference in Sharpe ratio. Although slightly driven by an impressive performance in the early 2000s and 2020s, pooled MLTSFM has consistently performed well.

5.8 Mean-Variance Analysis

Mean-variance optimization provides a natural framework for evaluating whether the momentum strategies studied in this thesis offer diversification benefits beyond what is already available from standard risk factors. Rather than assessing each strategy in isolation, mean-variance optimization asks whether a rational investor with a given level of risk aversion would allocate any weight to these strategies when forming an optimal portfolio. In the same way as Gupta and Kelly (2019), we include the TSFM, MLTSFM, and MLCSFM strategies alongside a set of benchmark factors and examine the sign and statistical significance of their allocations in the tangency portfolio. We will also do this for the pooled TSFM/MLTSFM strategies. For any strategy to receive weight in the tangency portfolio it needs to offer a return profile that is not fully spanned by the other assets. In other words, a strategy must offer incremental risk-adjusted return after accounting for its correlations with the other assets. We therefore begin by looking at the correlation structure across strategies.

5.8.1 Correlations

Table 10 reports the correlations of the MLTSFM and MLCSFM ensemble strategies with a set of standard benchmarks. Several observations are worth noticing. To begin with, all strategies exhibit low and positive correlations with the UMD factor, ranging from 0.06 to 0.11 for MLTSFM and 0.00 to 0.04 for MLCSFM. This is consistent with the alpha results discussed earlier, confirming that the momentum strategies capture a return component that is largely distinct from standard stock-level momentum. The correlations with the short-term reversal factor (STR) are negative across all strategies, ranging from -0.51 to -0.58 . The values for TSFM and CSFM are moderate and positive, in the range of 0.68 to 0.77, suggesting that while the ML strategies are related to their respective baselines they are not redundant. The correlation with the equal-weighted factor portfolio (EW) is relatively low, ranging from 0.07 to 0.29, suggesting that the timing signal generates return variation that goes beyond passive factor exposure. Note that the correlations with EW are lower for MLTSFM compared to MLCSFM for all ensembles. Finally, the correlation between the MLTSFM and MLCSFM ensemble strategies is notably high, ranging from 0.82 to 0.87, suggesting that the two frameworks tend to take broadly similar positions over time.

This is not very surprising as both approaches have the same underlying factor universe and factors that perform well in absolute terms tend to also rank highly in relative terms.

Table 10: Correlations of MLTSFM/MLCSFM Ensemble Strategies With Benchmarks.

Strategy	MLTSFM					MLCSFM					MLTSFM & MLCSFM
	UMD	STR	TSFM	CSFM	EW	UMD	STR	TSFM	CSFM	EW	
Ens	0.11	-0.56	0.72	0.72	0.13	0.01	-0.54	0.72	0.73	0.28	0.84
Ens-E	0.06	-0.55	0.68	0.67	0.07	0.00	-0.58	0.76	0.77	0.26	0.82
Ens-F	0.10	-0.52	0.68	0.68	0.15	0.04	-0.51	0.66	0.67	0.29	0.87

Table 11 examines the correlations of the pooled MLTSFM ensemble strategies with the same set of benchmarks, with the addition of the MLTSFM and MLCSFM expanding ensembles as reference points. The UMD and STR correlations remain consistent with those observed in the previous table. The correlations with TSFM and CSFM are again moderate, in the range of 0.56 to 0.58, slightly lower than the corresponding values in table 10. Most notably, the correlation between the pooled MLTSFM strategies and the MLCSFM expanding ensemble is high, reaching 0.74–0.81. This indicates that the strategies take similar positions over time. The correlations with the equal-weighted factor portfolio (EW) are higher here at around 0.63–0.67 compared to the individual MLTSFM and MLCSFM strategies, suggesting that the pooled approach is more aligned with broad factor exposure, something we already suspected to be the case in the initial analysis of pooled MLTSFM.

Table 11: Correlations of Pooled MLTSFM Ensemble Strategies with Benchmarks.

Strategy	Pooled MLTSFM							
	UMD	STR	TSFM	CSFM	MLTSFM Ens-E	MLCSFM Ens-E	EW	
Ens	0.1036	-0.4587	0.5832	0.5793	0.6580	0.7991	0.6690	
Ens-E	0.0985	-0.4593	0.5734	0.5657	0.6888	0.8146	0.6727	
Ens-F	0.1044	-0.4359	0.5618	0.5602	0.5983	0.7437	0.6269	

5.8.2 Moving Block Bootstrap

To assess the statistical reliability of the mean-variance portfolio weights, we use a technique called moving block bootstrapping. Standard bootstrapping methods can only be used when observations are independent and identically distributed (IID). This requirement is often violated when working with time series data which can contain serial correlations, and in our case we know that the data is not IID

from section 5.2. Block bootstrapping addresses this issue by sampling the data in overlapping windows of fixed length rather than individual data points. In this way, the autocorrelation structure in the return series is preserved (Künsch, 1989). Whilst taking care of autocorrelation structure is important, covariance structure is also essential. This is particularly important in the context of mean-variance optimization, where the portfolio weights are highly sensitive to the estimated covariance matrix. By re-sampling overlapping blocks rather than individual observations, the moving block bootstrap preserves the cross-sectional covariance relationships between strategies within each block. In each bootstrap iteration, we resample blocks⁵ of returns from the original time series, recompute the mean-variance optimal portfolio weights, and record the allocation to each strategy. By repeating this procedure many times — 2000 in our case — we get an empirical distribution of portfolio weights from which we can construct confidence intervals. Even though the bootstrap procedure may allow us to gauge roughly if parameter estimates are significant, it is not a perfect method, and large emphasis should not be placed on whether a parameter estimate is significant or not, but rather on the sign and to some degree size of them.

5.8.3 Tangency Portfolios

Table 12: Mean-Variance Optimal Weights Across Portfolio Specifications

Strategy	Indiv. SR	1	2	3	4	5	6	7	8	9	10	11	12	13
MLTSFM Ens	0.85	0.16			0.01	0.04								
MLTSFM Ens-E	0.86	0.58			0.14*	0.15**	0.82**		0.42		-0.28			
MLTSFM Ens-F	0.79	0.25			0.05	0.06								
MLCSFM Ens	0.83		0.70											
MLCSFM Ens-E	0.89		0.87					1.21**	0.58*			-0.83*		
MLCSFM Ens-F	0.69		-0.57											
Pooled MLTSFM Ens	1.44		-10.33**											
Pooled MLTSFM Ens-E	1.56			5.85**						1.23**	1.28**	1.83**	0.77**	0.48**
Pooled MLTSFM Ens-F	1.31			5.48**										
EW Factor	1.26												0.23	
TSFM	0.68						0.18			-0.23				
CSFM	0.62							-0.21						
UMD	0.42				0.22**	0.18**								0.15**
STR	0.23					0.16**								0.18**
MKT	0.56				0.20**	0.13**								0.20**
SMB	0.04				0.08*	0.05								0.06
HML	0.12				0.13	0.07								0.05
RMW	0.68				0.16**	0.13**								-0.00
CMA	0.32				0.02	0.03								-0.13*
Sharpe		0.90	0.91	1.72	1.63	1.81	0.87	0.90	0.92	1.58	1.58	1.69	1.58	2.40

* and ** indicate that estimates are statistically significantly different from 0 at the 5% and 1% level respectively. Empty cells indicate the strategy is not included in the given portfolio specification.

Table 12 reports the ex-post mean-variance optimal weights for each strategy across the 13 portfolio specifications along with the individual Sharpe ratio of each strategy and the Sharpe ratio of the resulting

⁵Each block is 12 months.

tangency portfolio. Each strategy/factor is scaled ex-post to have an annual volatility of 10% for a fairer comparison.

For portfolios 1 and 2, which consist solely of MLTSFM and MLCFSM ensemble strategies, respectively, the optimizer assigns weights to multiple strategies within each group, though all are statistically insignificant. More interesting are the weights in portfolio 3. The Pooled MLTSFM Ens receives a large and highly significant negative weight of -10.33, while Pooled MLTSFM Ens-E and Ens-F both receive significant positive weights of 5.85 and 5.48 respectively. This pattern of large offsetting positive and negative weights is typical of mean-variance optimization when assets are highly correlated, as the optimizer exploits small differences in return and risk profiles by taking large long-short positions. Given that the pooled MLTSFM strategies share the same underlying factor universe and differ only in their window scheme, it is reasonable to expect high correlations between them. The resulting tangency portfolio achieves a Sharpe ratio of 1.72, substantially higher than any individual strategy.

Portfolio 4 adds the FF5 factors and UMD to the MLTSFM strategies while Portfolio 5 adds these in addition to the STR factor. MLTSFM Ens-E receives positive and significant weight in both specifications, with values of 0.14 and 0.15 respectively. The other two MLTSFM models receive lower (and insignificant) weight while the Fama-French factors absorb some of the weight. Nevertheless, the tangency portfolio Sharpe ratios of 1.63 and 1.81 remain high, confirming that the combined portfolio continues to benefit from the diversification across momentum and risk factor strategies. Portfolio 6 consists of only MLTSFM Ens-E and TSFM while Portfolio 7 combines MLCFSM Ens-E and CSFM. In both specifications, the ML-model dominates the baseline strategy, receiving large and significant weight. An interesting result is that MLCFSM Ens-E receives positive and significant weight when combined with only MLTSFM Ens-E, whose weight is insignificant.

Portfolios 9 through 13, which are built around the Pooled MLTSFM Ens-E, are particularly informative. When combined with TSFM (Portfolio 9), MLTSFM Ens-E (Portfolio 10), MLCFSM Ens-E (Portfolio 11), or the EW factor portfolio (Portfolio 12), the Pooled MLTSFM Ens-E consistently receives significant positive weights, with the additional asset having insignificant weight in most cases. Notice that for Portfolio 9, 10 and 11 we see a negative weight on TSFM, MLTSFM and MLCFSM. This is as expected due to them having a negative alpha when regressed on pooled MLTSFM. Portfolio 13, which combines Pooled MLTSFM Ens-E with the full set of UMD, STR, and FF5 factors, achieves the highest tangency Sharpe ratio of 2.40. The weights are positive and significant for some of the factors as well, but Pooled MLTSFM Ens-E clearly receives the highest weight at 0.48, the second highest being the MKT factor at 0.20. This strongly suggests that the Pooled MLTSFM Ens-E offers incremental risk-adjusted return that is not spanned by standard risk factors or traditional momentum strategies.

Expanding our analysis to add a no-short selling constraint we can see much of the same story as before, without the extreme weights we experienced in portfolio 3. These results are presented in table 13.

Table 13: Mean-Variance Optimal Weights Across Portfolio Specifications (No-Short Selling)

Strategy	Indiv. SR	1	2	3	4	5	6	7	8	9	10	11	12	13
MLTSFM Ens	0.85	0.16			0.01	0.04								
MLTSFM Ens-E	0.86	0.58**			0.14**	0.15**	0.82**		0.42*		0.00			
MLTSFM Ens-F	0.79	0.25			0.05	0.06								
MLCSFM Ens	0.83		0.09											
MLCSFM Ens-E	0.89		0.91**					1.00**	0.58*			0.00		
MLCSFM Ens-F	0.69		0.00											
Pooled MLTSFM Ens	1.44			0.00										
Pooled MLTSFM Ens-E	1.56			1.00**						1.00**	1.00**	1.00**	0.77**	0.40**
Pooled MLTSFM Ens-F	1.31			0.00										
EW Factor	1.26												0.23*	
TSFM	0.68						0.18			0.00				
CSFM	0.62							0.00						
UMD	0.42				0.22**	0.18**								0.14**
STR	0.23					0.16**								0.18**
MKT	0.56				0.20**	0.13**								0.21**
SMB	0.04				0.08*	0.05*								0.05*
HML	0.12				0.13*	0.07								0.00
RMW	0.68				0.16**	0.13**								0.02
CMA	0.32				0.02	0.03								0.00
Sharpe		0.90	0.89	1.56	1.63	1.81	0.87	0.89	0.92	1.56	1.56	1.56	1.58	2.33

* and ** indicate that estimates are statistically significantly greater than 0 at the 5% and 1% level respectively. Empty cells indicate the strategy is not included in the given portfolio specification.

The portfolios that differ between tables 12 and 13 are specifications 2, 3, 7, 9, 10, 11, and 13. We clearly see the dominance of the expanding ensemble portfolio in specification 2 and 3, as the other ensembles receive close to zero or exactly zero weight. For specification 7, 9, 10, and 11, the assets with negative alpha when regressed on the other asset (all of these specifications are 2 asset portfolios) in the portfolio receive exactly zero weight, which is expected as a positive weight would have resulted in a lower Sharpe. Portfolio 13 is quite similar to before, with the major change being CMA going from a negative to zero weight, but pooled MLTSFM is still dominant within the portfolio. Sharpe ratios for the altered portfolios do marginally decrease, with the largest decrease being in portfolio 3, going from 1.72 to 1.56, but this result should be interpreted cautiously as the unconstrained specification had very extreme and unrealistic weights.

Overall, the mean-variance results support the conclusion that the ML-based factor momentum strategies, and particularly the pooled MLTSFM expanding ensemble, contain valuable timing information that a rational investor would want to include in an optimal portfolio. The consistently high tangency Sharpe ratios across specifications further reinforce the economic significance of these strategies beyond what can be achieved with only standard benchmarks.

5.9 Turnover & transaction cost analysis

Factor portfolios can experience very high turnover depending on what signal is used to construct them, and how often that signal is observed. The importance of this analysis is highlighted when talking about factors such as UMD and STR, as they both use the same type of signal (return), but the turnover of STR is much higher. In our case the signal for both of them is also observed on a monthly basis. Last month's return observed every month is going to fluctuate much more than the last 2-12 months return observed every month, leading to much larger changes in portfolio construction for STR. This example highlights why turnover & transaction cost analysis is essential for understanding how factors compare. Figure 14 presents turnover, gross, and net Sharpe for a range of momentum and factor momentum portfolios. Note that pooled MLTSFM is not included in this initial analysis. STR and UMD are chosen as baselines for high and low turnover momentum strategies. To compute turnover and average yearly turnover we use equations 12 and 13. We acknowledge that this does not account for weight drift as individual stocks earn returns over a month, but we still consider this a reasonable approximation. $w_{i,t}$ is the weight of stock i in time period t . We do not divide turnover by 2 due to transaction costs being incurred on the purchase and sale of a stock. Transaction cost computations assume a cost of 10 bps per unit of turnover, the same as used by Gupta and Kelly (2019) based on estimates from Frazzini et al. (2012).

$$TO_t = \sum_{i=1}^N |w_{i,t} - w_{i,t-1}|, \quad (12)$$

$$\overline{TO} = \frac{12}{T} \sum_{t=1}^T \sum_{i=1}^N |w_{i,t} - w_{i,t-1}|. \quad (13)$$

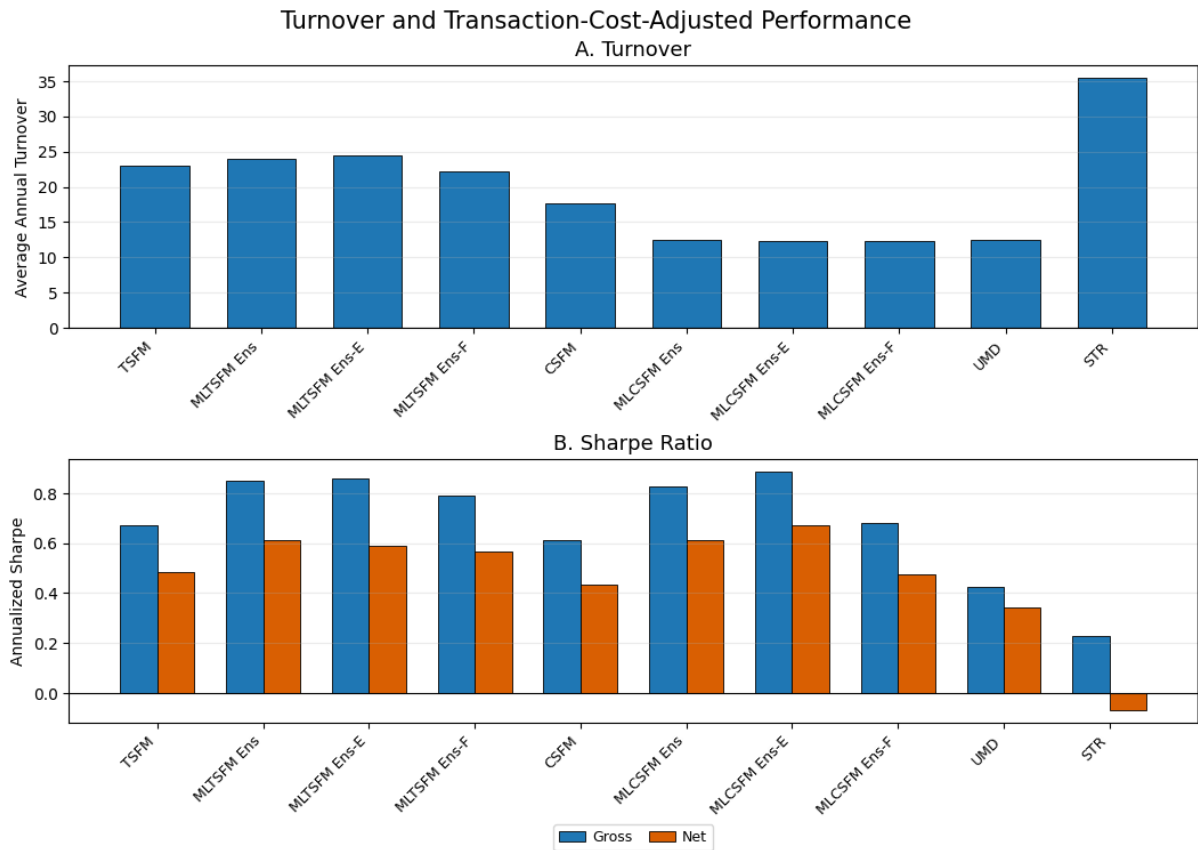


Figure 14

While all variants of TSFM and CSFM are based on one month return measures like STR, which are highly variable, by combining factors into portfolios some of the stock weights will end up netting each other out, which in turn leads to lower gross stock exposure after taking this into account. This lower gross stock exposure in turn lowers turnover, which at the extreme end results in MLCFSM portfolios having turnover comparable to UMD, a factor created with a much more stable signal. Table 14 presents the effective stock exposures after netting of weights is taken into account. We can see that the single factor portfolios do not experience weights netting each other out, as expected.

Table 14: Effective stock-level exposure summary after netting

Strategy	Long mean	Short mean	Gross mean	Gross min	Gross max
TSFM	0.7552	-0.7552	1.5104	0.9299	2.0385
MLTSFM Ens	0.8622	-0.8622	1.7245	0.7668	2.3739
MLTSFM Ens-E	0.8646	-0.8646	1.7292	0.6749	2.4106
MLTSFM Ens-F	0.8167	-0.8167	1.6334	0.7668	2.4459
CSFM	0.5790	-0.5790	1.1580	0.6452	1.5387
MLCSFM Ens	0.4698	-0.4698	0.9396	0.5681	1.4184
MLCSFM Ens-E	0.4518	-0.4518	0.9037	0.5681	1.3547
MLCSFM Ens-F	0.4736	-0.4736	0.9472	0.5681	1.3547
UMD	1.0000	-1.0000	2.0000	2.0000	2.0000
STR	1.0000	-1.0000	2.0000	2.0000	2.0000

The resulting decrease in Sharpe after taking into account transaction costs is most devastating for STR, as it completely wipes out its performance and actually brings it into negative territory. TSFM still ends up slightly beating CSFM, whilst most ML portfolios end up around the same Sharpe of 0.60, with the exception of MLCSFM Ens-E who still has the best performance, and MLCSFM Ens-F who still has the worst performance of normal ML portfolios. The normal ML portfolios still end up beating their TSFM/CSFM counterparts when looking at net Sharpe.

Expanding our analysis to include the pooled portfolios, figure 15 presents turnover and gross/net Sharpe for the EW factor portfolio, and all three pooled MLTSFM ensembles.

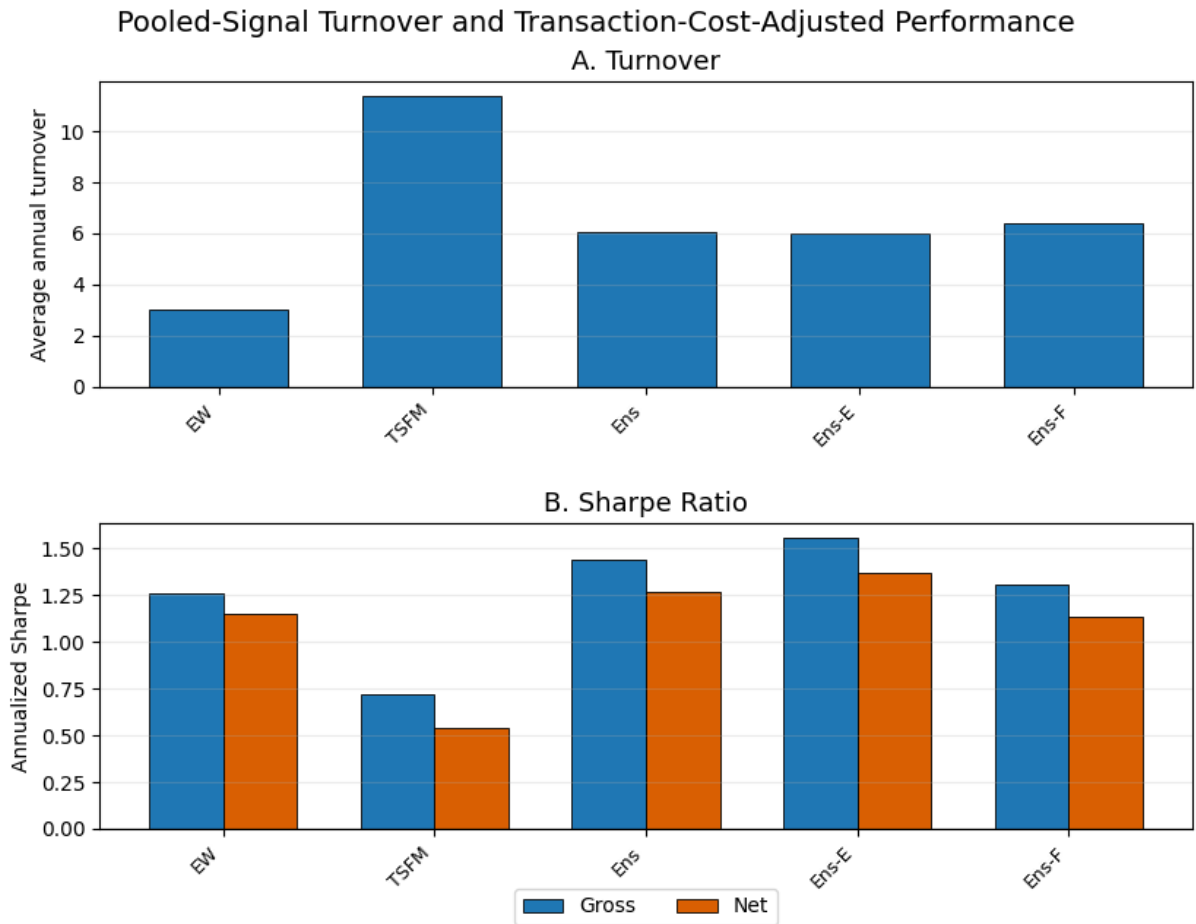


Figure 15

Note that since normal TSFM/CSFM portfolios are constructed with a \$1 long and \$1 short side, their gross factor exposure is 2 most of the time when both legs are present. By contrast, pooled TSFM portfolios and the EW portfolio have a gross factor exposure of 1, which partly contributes to lower turnover. We can see a large difference in the turnover between pooled TSFM and MLTSFM, with pooled TSFM having nearly double the turnover. This is not as surprising when compared to the EW portfolio, as we have already seen that pooled MLTSFM creates portfolios that are more similar to the EW factor portfolio. After accounting for transaction costs we continue to see pooled MLTSFM being very impressive from a Sharpe perspective, having slightly higher than double the Sharpe of normal MLTSFM portfolios. The net Sharpe of the EW portfolio is of course still impressive.

Some of the lower turnover of pooled MLTSFM can be explained by the effective gross stock exposures after taking into account weight netting, these are presented in table 15. All of the pooled variants have a gross stock exposure under half of their normal counterparts. It must be noted that pooled TSFM only has a slightly higher gross exposure than pooled MLTSFM, whilst having much higher turnover, which is most likely explained by pooled TSFM having a larger variation in net factor bias seen in table 8.

Table 15: Effective stock-level exposure summary after netting for pooled portfolios

Strategy	Long mean	Short mean	Gross mean	Gross min	Gross max
EW Factor	0.2736	-0.2736	0.5472	0.4492	0.6331
Pooled TSFM	0.3788	-0.3788	0.7577	0.4862	0.9936
Pooled MLTSFM Ens	0.3573	-0.3573	0.7146	0.5666	0.8679
Pooled MLTSFM Ens-E	0.3482	-0.3482	0.6963	0.5549	0.8679
Pooled MLTSFM Ens-F	0.3622	-0.3622	0.7244	0.5633	0.9510

This analysis shows that after accounting for transaction costs, the MLTSFM portfolios achieve the highest net Sharpe ratios, while the pooled MLTSFM portfolios remain the strongest-performing strategies overall.

6 Conclusion

Our thesis extends the factor momentum framework of [Gupta and Kelly \(2019\)](#) by replacing the linear momentum signal with predictions from gradient boosted trees, producing Machine-Learned Time-Series and Cross-Section Factor Momentum strategies (MLTSFM and MLCSFM) evaluated over the out-of-sample period 1985–2024.

We first replicate the core findings of [Gupta and Kelly \(2019\)](#), confirming that a large majority of our 48 factors exhibit positive autocorrelation and that the aggregated TSFM strategy achieves good risk-adjusted performance, with a Sharpe ratio of 0.68, an own-factor alpha of 7.05%, and a FF5 alpha of 7.49%. This provides the foundation for our machine learning extension.

Our main finding is that the MLTSFM strategies consistently match or exceed TSFM across all performance metrics. Sharpe ratios range from 0.69 to 0.89 compared to 0.68 for TSFM, and all strategies deliver significant own-factor alphas, and significant alphas relative to TSFM and CSFM. An important result is that the MLTSFM strategies exhibit substantially narrower confidence intervals than TSFM, reflecting more stable and statistically reliable performance, an insight which is valued by potential investors. The MLCSFM strategies deliver similar results, with Sharpe ratios ranging from 0.61 to 0.91 and positive own-factor alphas across all specifications, however not all are statistically significant. When comparing MLTSFM and MLCSFM, neither framework dominates the other across all metrics. MLTSFM tends to generate higher average returns and more consistently significant alphas, while MLCSFM occasionally achieves higher Sharpe ratios, particularly for expanding window models. Both strategies

deliver returns that are not explained by standard risk factors or traditional stock-level momentum. Expanding window models consistently outperform fixed window counterparts, and ensemble models in general perform competitively and often deliver among the strongest results, particularly by stabilizing performance across specifications.

We additionally introduce a pooled variant of TSFM/MLTSFM that allows for a net factor bias. The pooled MLTSFM strategies achieve gross Sharpe ratios up to 1.56 and remain strong after transaction costs. Ex-post mean-variance analysis confirms that the pooled MLTSFM expanding ensemble receives large and significant weight in tangency portfolios alongside standard risk factors, achieving a combined Sharpe ratio of 2.40 in the unconstrained specification. The pooled specification benefits both from machine-learned timing and from allowing a positive net factor exposure when forecasts are broadly positive.

Some limitations should be noted. First, the analysis is restricted to U.S. stocks and a factor universe of 48 characteristics, which limits direct comparability with the broader factor universe studied by [Gupta and Kelly \(2019\)](#). Second, the pooled MLTSFM results should be interpreted with some caution, since part of the strong performance is linked to positive net factor exposure rather than timing ability alone. Future research could extend the analysis to international markets, alternative factor universes, additional formation periods, and a broader set of machine learning models.

In conclusion, our results demonstrate that machine learning can meaningfully improve the timing of systematic equity factor returns, delivering more stable performance and a distinct source of return not fully captured by existing risk premia or traditional momentum strategies.

References

- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1):5–32.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.
- Cochrane, J. H. (2011). Presidential address: Discount rates. *The Journal of Finance*, 66(4):1047–1108.
- Fama, E. F. and French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1):3–56.
- Fama, E. F. and French, K. R. (2015). A five-factor asset pricing model. *Journal of Financial Economics*, 116(1):1–22.
- Frazzini, A., Israel, R., and Moskowitz, T. J. (2012). Trading costs of asset pricing anomalies. Fama-Miller Working Paper Chicago Booth Research Paper No. 14-05, Fama-Miller Center for Research in Finance.
- Frazzini, A. and Pedersen, L. H. (2014). Betting against beta. *Journal of Financial Economics*, 111(1):1–23.
- French, K. R. (2026). Data library. Accessed: 2026.
- Gu, S., Kelly, B., and Xiu, D. (2020). Empirical Asset Pricing via Machine Learning. *The Review of Financial Studies*, 33(5):2223–2273.
- Gupta, T. and Kelly, B. (2019). Factor Momentum Everywhere. *The Journal of Portfolio Management*, 45(3):13–36.
- Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2 edition.
- Jensen, T. I., Kelly, B., and Pedersen, L. H. (2023). Is there a replication crisis in finance? *The Journal of Finance*, 78(5):2465–2518.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.
- Kelly, B., Malamud, S., and Zhou, K. (2024). The Virtue of Complexity in Return Prediction. *The Journal of Finance*, 79(1):459–503.

- Künsch, H. R. (1989). The jackknife and the bootstrap for general stationary observations. *The Annals of Statistics*, 17:1217–1241.
- Moskowitz, T. J., Ooi, Y. H., and Pedersen, L. H. (2012). Time series momentum. *Journal of Financial Economics*, 104(2):228–250.
- Moskowitz, T. J., Sabbatucci, R., Tamoni, A., and Uhl, B. (2025). Nonlinear Time Series Momentum. Working paper, Social Science Research Network (SSRN). SSRN Scholarly Paper.
- Newey, W. K. and West, K. D. (1987). A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3):703–708.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31.
- Rasmussen, C. E. and Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press.
- Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., and de Freitas, N. (2016). Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104:148–175.
- Sharpe, W. F. (1964). Capital Asset Prices: A Theory of Market Equilibrium Under Conditions of Risk. *The Journal of Finance*, 19(3):425–442.
- Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25.